

Automated Version Control with Git and GitHub for Data Science

Best Practices for Research

Leykun G. and Mesfin D., DAVT-NDMC, EPHI

26-30 March, 2026

Automated Version Control

What is version control?

- A version control system (VCS) tracks changes to files over time, so you can recover earlier versions of a file or the whole project.
- It also lets multiple people work on the same project at the same time without overwriting each other's work.
- VCS is often used interchangeably with software configuration management (SCM), though version control is only one part of SCM.
- With version control, you can see who changed what and when, add commit messages, create branches for experimental work, merge changes back into the main branch, and tag releases.
- Git is a fast, scalable, free, open-source distributed VCS, created by Linus Torvalds in 2005.

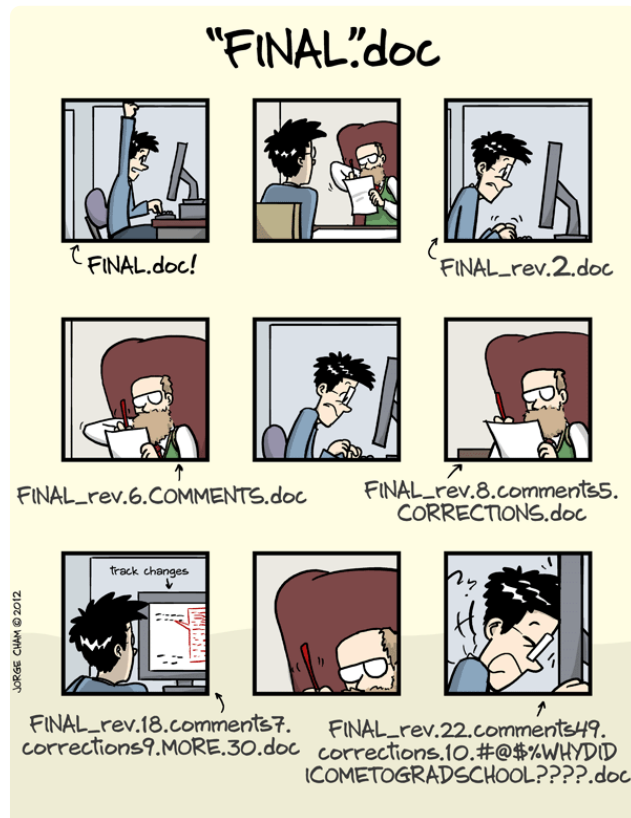
Distributed version control

- Older VCS tools like CVS, Subversion (SVN), and Perforce used a central server, which could become single point of failure.
- Git is distributed, so the full project history lives on both the client and the server; you can work offline and sync later.
- If the server fails, your local copy still contains the project history.

Git terminology

The Problem with Manual Version Control

- We'll start by exploring how version control can be used to keep track of what one person did and when. Even if you aren't collaborating with other people, automated version control is much better than this situation:



- We've all been in this situation before: it seems unnecessary to have multiple nearly-identical versions of the same document.
- Some word processors let us deal with this a better, such as Microsoft Word's Track Changes, Google Docs' version history, or LibreOffice's Recording and Displaying Changes.

How Version Control Works

- Version control systems start with a base version of the document and then record changes you make each step of the way. You can think of it as a recording of your progress: you can rewind to start at the base document and play back each change you made, eventually arriving at your more recent version



- Once you think of changes as separate from the document itself, you can then think about “playing back” different sets of changes on the base document, ultimately resulting in different versions of that document. For example, two users can make independent sets of changes on the same document.

Multiple Versions



What is a Version Control System?

- A version control system is a tool that keeps track of these changes for us, effectively creating different versions of our files. It allows us to decide which changes will be made to the next version (each record of these changes is called a commit), and keeps useful metadata about them, such as who made the change.
- The complete history of commits for a particular project and their metadata make up a repository. Repositories can be kept in sync across different computers, facilitating collaboration among different people.

The Long History of Version Control Systems

- Early version control (1980s): RCS, CVS, Subversion—now legacy due to limitations.
- Modern distributed systems: Git, Mercurial—no central server, strong merging for concurrent edits.
- Git's origin: Created by Linus Torvalds (2005) to replace BitKeeper for Linux kernel; name ideas in README (e.g., “Global Information Tracker”).

Challenge: Paper Writing

Consider these scenarios:

1. Imagine you drafted an excellent paragraph for a paper you are writing, but later ruin it. How would you retrieve the *excellent* version of your conclusion? Is it even possible?
2. Imagine you have 5 co-authors. How would you manage the changes and comments they make to your paper? If you use LibreOffice Writer or Microsoft Word, what happens if you accept changes made using the **Track Changes** option? Do you have a history of those changes?

► [View Solution](#)

Setting Up Git

Questions

- How do I get set up to use Git?

First-Time Git Configuration

- When we use Git on a new computer for the first time, we need to configure a few things:
 - our name and email address,
 - what our preferred text editor is,
 - and that we want to use these settings globally (i.e. for every project).
- On a command line, Git commands are written as `git verb options`, where `verb` is what we actually want to do and `options` is additional optional information which may be needed for the `verb`.

Setting Your Identity

Here is how Anwar sets up his new laptop:

```
1 $ git config --global user.name "Anwar Taju"  
2 $ git config --global user.email "awar.taju@ephi.gov.et"
```

- Please use your own name and email address instead of Anwar's. This user name and email will be associated with your subsequent Git activity, which means that any changes pushed to GitHub, BitBucket, GitLab or another Git host server after this lesson will include this information.
- For this lesson, we will be interacting with GitHub and so the email address used should be the same as the one used when setting up your GitHub account.

Keeping your email private

- If you elect to use a private email address with GitHub, then use GitHub's no-reply email address for the `user.email` value. It looks like `ID+username@users.noreply.github.com`. You can look up your own address in your GitHub email settings.

Line Endings

- As with other keys, when you press Enter or Return on your keyboard, your computer encodes this input as a character. Different operating systems use different character(s) to represent the end of a line (You may also hear these referred to as newlines or line breaks.)
- Because Git uses these characters to compare files, it may cause unexpected issues when editing a file on different machines.
- You can change the way Git recognizes and encodes line endings using the `core.autocrlf` command to `git config`.

Recommended Line Ending Settings

On macOS and Linux:

```
1 $ git config --global core.autocrlf input
```

On Windows:

```
1 $ git config --global core.autocrlf true
```

You can read more about this issue in the Pro Git book.

Setting Your Text Editor

Anwar also has to set his favorite text editor:

Editor	Configuration command
Atom	<code>git config --global core.editor "atom --wait"</code>
nano	<code>git config --global core.editor "nano -w"</code>
BBEdit (Mac)	<code>git config --global core.editor "bbedit -w"</code>
Sublime Text (Mac)	<code>git config --global core.editor "/Applications/Sublime\ Text.app/Contents/SharedSupport/bin/subl -n -w"</code>
Sublime Text (Win, 32-bit)	<code>git config --global core.editor "'c:/program files (x86)/sublime text 3/sublime_text.exe' -w"</code>
Sublime Text (Win, 64-bit)	<code>git config --global core.editor "'c:/program files/sublime text 3/sublime_text.exe' -w"</code>
Notepad (Win)	<code>git config --global core.editor "c:/Windows/System32/notepad.exe"</code>

- It is possible to reconfigure the text editor for Git whenever you want to change it.

Exiting Vim

- Note that Vim is the default editor for many programs. If you haven't used Vim before and wish to exit a session without saving your changes, press `Esc` then type `:q!` and press Enter. If you want to save your changes and quit, press `Esc` then type `:wq` and press Enter.

Setting the Default Branch Name

- Git (2.28+) allows configuration of the name of the branch created when you initialize any new repository. Anwar decides to use that feature to set it to `main` so it matches the cloud service he will eventually use.

```
1 $ git config --global init.defaultBranch main
```

Default Git Branch Naming

- Source file changes are associated with a “branch.” For new learners in this lesson, it’s enough to know that branches exist, and this lesson uses one branch.
- By default, Git will create a branch called `master` when you create a new repository with `git init`. This term evokes the racist practice of human slavery and the software development community has moved to adopt more inclusive language.
- In 2020, most Git code hosting services transitioned to using `main` as the default branch. As an example, any new repository that is opened in GitHub and GitLab default to `main`. However, Git has not yet made the same change. As a result, local repositories must be manually configured to have the same main branch name as most cloud services.
- For versions of Git prior to 2.28, the change can be made on an individual repository level. Note that if this value is unset in your local Git configuration, the `init.defaultBranch` value defaults to `master`.

Viewing and Testing Your Configuration

- The five commands we just ran above only need to be run once: the flag `--global` tells Git to use the settings for every project, in your user account, on this computer.
- Let's review those settings and test our `core.editor` right away:

```
1 $ git config --global --edit
```

- Let's close the file without making any additional changes. Remember, since typos in the config file cause issues, it's safer to view the configuration with:

```
1 $ git config --list --global
```

- And if necessary, change your configuration using the same commands to choose another editor or update your email address. This can be done as many times as you want.

Proxy Settings

Proxy Configuration

- In some networks you need to use a proxy. If this is the case, you may also need to tell Git about the proxy:

```
1 $ git config --global http.proxy proxy-url
2 $ git config --global https.proxy proxy-url
```

- To disable the proxy, use:

```
1 $ git config --global --unset http.proxy
2 $ git config --global --unset https.proxy
```

Git Help and Manual

- Always remember that if you forget the subcommands or options of a `git` command, you can access the relevant list of options by typing `git <command> -h` or access the corresponding Git manual by typing `git <command> --help`, e.g.:

```
1 $ git config -h
2 $ git config --help
```

- While viewing the manual, remember the `:` is a prompt waiting for commands and you can press `Q` to exit the manual.
- More generally, you can get the list of available `git` commands and further resources of the Git manual by typing:

```
1 $ git help
```

Creating a Repository

Creating a New Repository

- Once Git is configured, we can start using it.
- We will help Anwar with his new project, create a repository with all his project.

First, let's create a new directory in the **Desktop** folder for our work and then change the current working directory to the newly created one:

```
1 $ cd ~/Desktop
2 $ mkdir Malaria
3 $ cd Malaria
```

Then we tell Git to make **Malaria** a repository — a place where Git can store versions of our files:

```
1 $ git init
```

Understanding `git init`

- It is important to note that `git init` will create a repository that can include subdirectories and their files — There is no need to create separate repositories nested within the `Malaria` repository, whether subdirectories are present from the beginning or added later.
- Also, note that the creation of the `Malaria` directory and its initialization as a repository are complete separate processes.
- If we use `ls` to show the directory's contents, it appears that nothing has changed:

```
1 $ ls
```

- But if we add the `-a` flag to show everything, we can see that Git has created a hidden directory within `Malaria` called `.git`:

```
1 $ ls -a
```

```
1 .  ..  .git
```

The .git Directory

- Git uses this special subdirectory to store all the information about the project, including the tracked files and sub-directories located within the project's directory. If we ever delete the `.git` subdirectory, we will lose the project's history.
- We can now start using one of the most important git commands, which is particularly helpful to beginners. `git status` tells us the status of our project, and better, a list of changes in the project and options on what to do with those changes. We can use it as often as we want, whenever we want to understand what is going on.

```
1 $ git status
```

```
1 On branch main
2
3 No commits yet
4
5 nothing to commit (create/copy files and use "git add" to track)
```

- If you are using a different version of `git`, the exact wording of the output might be slightly different.

Challenge: Places to Create Git Repositories

- Along with tracking information about Malaria (the project we have already created), Anwar would also like to track information about eda specifically.
- Anwar creates a `eda` project inside his `Malaria` project with the following sequence of commands:

```
1 $ cd ~/Desktop      # return to Desktop directory
2 $ cd Malaria        # go into Malaria directory, which is already a Git repository
3 $ ls -a             # ensure the .git subdirectory is still present in the Malaria directory
4 $ mkdir eda         # make a sub-directory Malaria/eda
5 $ cd eda            # go into eda subdirectory
6 $ git init          # make the eda subdirectory a Git repository
7 $ ls -a             # ensure the .git subdirectory is present
```

- Is the `git init` command, run inside the `eda` subdirectory, required for tracking files stored in the `eda` subdirectory?

► View Solution

Correcting “git init” Mistakes

Mesfin explains to Anwar how a nested repository is redundant and may cause confusion down the road. Anwar would like to go back to a single git repository. How can Anwar undo his last `git init` in the `ec` subdirectory?

► [View Solution](#)

Tracking Changes

Objectives

- Go through the modify-add-commit cycle for one or more files.
- Explain where information is stored at each stage of that cycle.
- Distinguish between descriptive and non-descriptive commit messages.

Questions

- How do I record changes in Git?
- How do I check the status of my version control repository?
- How do I record notes about what changes I made and why?

Creating a File

First let's make sure we're still in the right directory. You should be in the [Malaria](#) directory.

```
1 $ cd ~/Desktop/Malaria
```

- Let's create a file called [Malaria_incidence_analysis.py](#) that contains the basic structure of our Malaria analysis script. We'll use [nano](#) to edit the file; you can use whatever editor you like.

```
1 $ nano Malaria_incidence_analysis.py
```

Type the text below into the [Malaria_incidence_analysis.py](#) file:

```
1 # Malaria Incidence Analysis - Ethiopia
2
3 import pandas as pd
4 import numpy as np
5
6 def load_Malaria_data(filepath):
7     """Load Malaria incidence data from CSV file."""
8     data = pd.read_csv(filepath)
9     return data
10
11 def calculate_incidence(cases, population):
12     """Calculate Malaria incidence rate per 100,000 population."""
13     incidence = (cases / population) * 100000
14     return incidence
```

Save the file and exit your editor.

Python Reference

- Anwar and his colleagues at the Ethiopian Public Health Institute are using Python for their data analysis. Python is a powerful programming language for data science, with libraries like pandas for data manipulation and matplotlib for visualization.

Python Code	Description
<code>import pandas as pd</code>	Import pandas library for data analysis
<code>pd.read_csv('file.csv')</code>	Read CSV file into a DataFrame
<code>df.head()</code>	View first 5 rows of data
<code>df['column'].mean()</code>	Calculate mean of a column

Checking File Creation

Let's verify that the file was properly created by running the list command (`ls`):

```
1 $ ls
```

```
1 Malaria_incidence_analysis.py
```

`Malaria_incidence_analysis.py` contains the functions we defined, which we can see by running:

```
1 $ cat Malaria_incidence_analysis.py
```

```
1 # Malaria Incidence Analysis - Ethiopia
2
3 import pandas as pd
4 import numpy as np
5
6 def load_Malaria_data(filepath):
7     """Load Malaria incidence data from CSV file."""
8     data = pd.read_csv(filepath)
9     return data
10
11 def calculate_incidence(cases, population):
12     """Calculate Malaria incidence rate per 100,000 population."""
13     incidence = (cases / population) * 100000
14     return incidence
```

Checking Git Status

- If we check the status of our project again, Git tells us that it's noticed the new file:

```
1 $ git status
```

```
1 On branch main
2
3 No commits yet
4
5 Untracked files:
6   (use "git add <file>..." to include in what will be committed)
7
8   Malaria_incidence_analysis.py
9
10 nothing added to commit but untracked files present (use "git add" to track)
```

- The “untracked files” message means that there’s a file in the directory that Git isn’t keeping track of.

Adding and Committing

We can tell Git to track a file using `git add`:

```
1 $ git add Malaria_incidence_analysis.py
2 $ git status
```

```
1 On branch main
2
3 No commits yet
4
5 Changes to be committed:
6   (use "git rm --cached <file>..." to unstage)
7
8    new file:   Malaria_incidence_analysis.py
```

- Git now knows that it's supposed to keep track of `Malaria_incidence_analysis.py`, but it hasn't recorded these changes as a commit yet. To get it to do that:

```
1 $ git commit -m "Create initial Malaria incidence analysis script"
```

```
1 [main (root-commit) f22b25e] Create initial Malaria incidence analysis script
2 1 file changed, 12 insertions(+)
3 create mode 100644 Malaria_incidence_analysis.py
```

Understanding Commits

- When we run `git commit`, Git takes everything we have told it to save by using `git add` and stores a copy permanently inside the special `.git` directory. This permanent copy is called a commit and its short identifier is `f22b25e`.
- We use the `-m` flag (for “message”) to record a short, descriptive, and specific comment. Good commit messages start with a brief (<50 characters) statement about the changes made. The message should complete the sentence “If applied, this commit will ”.

If we run `git status` now:

```
1 $ git status
```

```
1 On branch main
2 nothing to commit, working tree clean
```

Viewing History

- If we want to know what we've done recently, we can ask Git to show us the project's history using `git log`:

```
1 $ git log
1 commit f22b25e3233b4645dabd0d81e651fe074bd8e73b
2 Author: Anwar Taju <anwar.taju@ephi.gov.et>
3 Date: Thu Aug 22 09:51:46 2013 -0400
4
5 Create initial Malaria incidence analysis script
```

- `git log` lists all commits made to a repository in reverse chronological order.

Where Are My Changes?

- If we run `ls` at this point, we will still see just one file called `Malaria_incidence_analysis.py`. That's because Git saves information about files' history in the special `.git` directory so that our filesystem doesn't become cluttered.

Modifying a File

- Now suppose Anwar adds more information to the file:

```
1 $ nano Malaria_incidence_analysis.py
2 $ cat Malaria_incidence_analysis.py
```

```
1 # Malaria Incidence Analysis - Ethiopia
2
3 import pandas as pd
4 import numpy as np
5
6 def load_Malaria_data(filepath):
7     """Load Malaria incidence data from CSV file."""
8     data = pd.read_csv(filepath)
9     return data
10
11 def calculate_incidence(cases, population):
12     """Calculate Malaria incidence rate per 100,000 population."""
13     incidence = (cases / population) * 100000
14     return incidence
15
```

```
16 def age_standardize_incidence(data, standard_population):
17     """Calculate age-standardized Malaria incidence rates."""
18     age_adj_incidence = data['incidence'] * standard_population
19     return age_adj_incidence
```

- When we run `git status` now, it tells us that a file it already knows about has been modified:

```
1 git status
```

```
1 On branch main
2 Changes not staged for commit:
3   (use "git add <file>..." to update what will be committed)
4   (use "git restore <file>..." to discard changes in working directory)
5
6     modified:   Malaria_incidence_analysis.py
7
8 no changes added to commit (use "git add" and/or "git commit -a")
```

Viewing Changes with `git diff`

- It is good practice to always review our changes before saving them. We do this using `git diff`. This shows us the differences between the current state of the file and the most recently saved version:

```
1 git diff

1 diff --git a/Malaria_incidence_analysis.py b/Malaria_incidence_analysis.py
2 index df0654a..315bf3a 100644
3 --- a/Malaria_incidence_analysis.py
4 +++ b/Malaria_incidence_analysis.py
5 @@ -7,3 +7,7 @@ def load_Malaria_data(filepath):
6  def calculate_incidence(cases, population):
7     incidence = (cases / population) * 100000
8     return incidence
9  +
10 +def age_standardize_incidence(data, standard_population):
11 +    age_adj_incidence = data['incidence'] * standard_population
12 +    return age_adj_incidence
```

- The `+` marker shows where we added lines.

Committing Changes

- After reviewing our change, it's time to commit it:

```
1 git commit -m "Add age-standardization function for Malaria incidence"
```

```
1 On branch main
2 Changes not staged for commit:
3   (use "git add <file>..." to update what will be committed)
4
5     modified:   Malaria_incidence_analysis.py
6
7 no changes added to commit
```

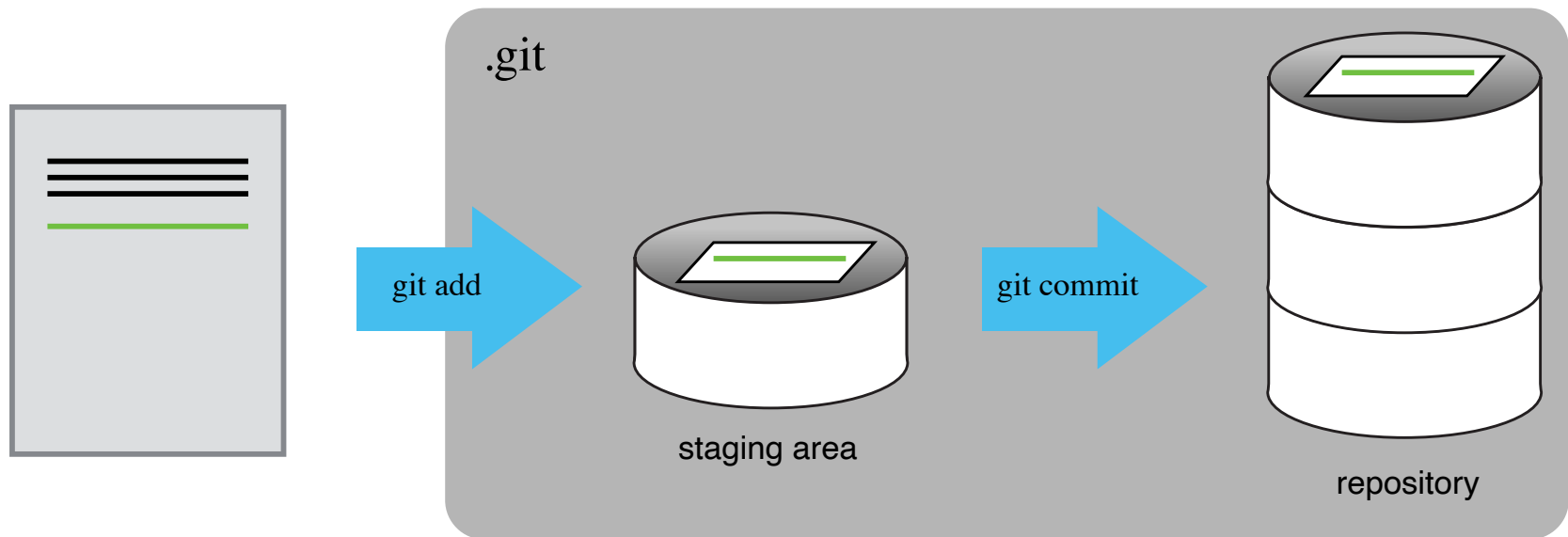
- Whoops: Git won't commit because we didn't use `git add` first. Let's fix that:

```
1 git add Malaria_incidence_analysis.py
2 git commit -m "Add age-standardization function for Malaria incidence"
```

```
1 [main 34961b1] Add age-standardization function for Malaria incidence
2 1 file changed, 4 insertions(+)
```

The Staging Area

- Git insists that we add files to the set we want to commit before actually committing anything. This allows us to commit our changes in stages and capture changes in logical portions.



Git Staging Area

Staging Area

- `git add` specifies *what* will go in a snapshot (putting things in the staging area), and `git commit` then *actually takes* the snapshot. Try to stage things manually, or you might find yourself searching for “git undo commit” more than you would like!

Making More Changes

Let's improve our analysis by adding data validation:

```
1 nano Malaria_incidence_analysis.py
2 cat Malaria_incidence_analysis.py
```

```
1 # Malaria Incidence Analysis - Ethiopia
2
3 import pandas as pd
4 import numpy as np
5
6 def load_Malaria_data(filepath):
7     """Load Malaria incidence data from CSV file."""
8     data = pd.read_csv(filepath)
9     return data
10
11 def validate_data(data):
12     """Check if required columns exist in the data """
```

Viewing Changes with `git diff` (Continued)

```
1 git diff
```

```
1 diff --git a/Malaria_incidence_analysis.py b/Malaria_incidence_analysis.py
2 index 315bf3a..b36abfd 100644
3 --- a/Malaria_incidence_analysis.py
4 +++ b/Malaria_incidence_analysis.py
5 @@ -4,6 +4,12 @@ import pandas as pd
6  import numpy as np
7
8  def load_Malaria_data(filepath):
9  +     """Load Malaria incidence data from CSV file."""
10     data = pd.read_csv(filepath)
11     return data
12
13 +def validate_data(data):
14  +     """Check if required columns exist in the data."""
15  +     required_cols = ['region', 'cases', 'population', 'year']
16  +     for col in required_cols:
17  +         if col not in data.columns:
18  +             raise ValueError(f"Missing required column: {col}")
19  +     return True
20  +
21  def calculate_incidence(cases, population):
22  +     """Calculate Malaria incidence rate per 100,000 population."""
23  +     incidence = (cases / population) * 100000
```

Viewing Staged Changes

Now let's put that change in the staging area:

```
1 git add Malaria_incidence_analysis.py
2 git diff
```

There is no output: as far as Git can tell, there's no difference between what it's been asked to save perm and what's currently in the directory.

However, if we do this:

```
1 git diff --staged
```

This shows us the difference between the last committed change and what's in the staging area.

Saving and Viewing History

Let's save our changes:

```
1 git commit -m "Add data validation function for Malaria analysis"
```

```
1 [main 005937f] Add data validation function for Malaria analysis
2 1 file changed, 7 insertions(+)
```

Check our status:

```
1 git status
```

```
1 On branch main
2 nothing to commit, working tree clean
```

And look at the history:

```
1 git log --oneline
```

```
1 005937f (HEAD -> main) Add data validation function for Malaria analysis
2 34961b1 Add age-standardization function for Malaria incidence
3 f22b25e Create initial Malaria incidence analysis script
```

Useful Log Options

Paging the Log

When the output of `git log` is too long to fit in your screen, Git uses a program to split it into pages. To get out pager, press `Q`. To move to the next page, press `Spacebar`.

To limit the number of commits:

```
1 git log -1
```

To see a more compact view:

```
1 git log --oneline
```

Combine with `--graph` to see the branch structure:

```
1 git log --oneline --graph
```

```
1 * 005937f (HEAD -> main) Add data validation function for Malaria analysis
2 * 34961b1 Add age-standardization function for Malaria incidence
3 * f22b25e Create initial Malaria incidence analysis script
```

Directories in Git

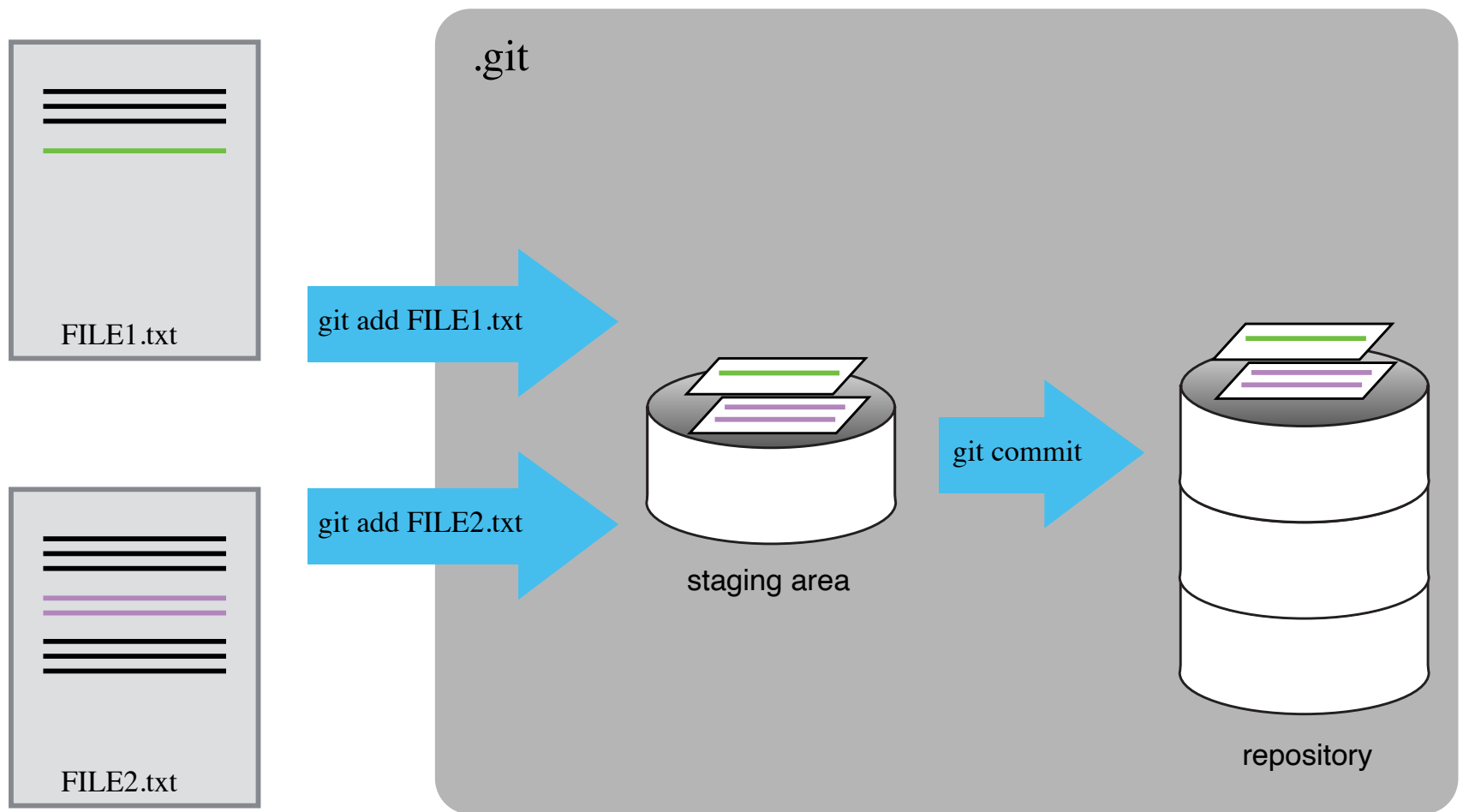
Important Facts About Directories

1. **Git does not track directories on their own**, only files within them. This is why you will sometimes see `.gitkeep` files in otherwise empty directories.
2. **You can add all files in a directory at once** by referring to the directory in your `git add` command:

```
1 mkdir data
2 touch data/ethiopia_Malaria_2020.csv data/ethiopia_Malaria_2021.csv
3 git add data
4 git status
5 git commit -m "Add Malaria data files"
```

The Commit Workflow

- To recap, when we want to add changes to our repository, we first need to add the changed files to the staging area (`git add`) and then commit the staged changes to the repository (`git commit`):



Git Committing

Challenge: Choosing a Commit Message

Which of the following commit messages would be most appropriate for adding the age-standardization function?

1. "Changes"
2. "Add function"
3. "Add age-standardization function for Malaria incidence analysis"

► View Solution

Challenge: Committing Changes to Git

Which command(s) below would save the changes of `Malaria_incidence_analysis.py` to your local Git repository?

1. `git commit -m "Update Malaria analysis"`
2. `git init Malaria_incidence_analysis.py` followed by `git commit -m "Update"`
3. `git add Malaria_incidence_analysis.py` followed by `git commit -m "Add data validation"`
4. `git commit -m Malaria_incidence_analysis.py "Update"`

► View Solution

Challenge: Committing Multiple Files

The staging area can hold changes from any number of files that you want to commit as a single snapshot.

1. Add a new function to `Malaria_incidence_analysis.py` for visualizing Malaria trends.
2. Create a new file `malaria_analysis.py` with basic functions for malaria analysis.
3. Add changes from both files to the staging area, and commit those changes.

► View Solution

Exploring History

Understanding HEAD

- **HEAD** refers to the most recent commit of the current branch
- Can be used with commands like `git diff` and `git restore` to reference the latest committed state

```
1 git diff HEAD eda/Malaria_incidence_analysis.py
```

Comparing with Previous Commits

- Use `HEAD~1`, `HEAD~2`, etc., to refer to earlier commits

```
1 git diff HEAD~1 Malaria_incidence_analysis.py
2 git diff HEAD~2 Malaria_incidence_analysis.py
```

- `git show` displays changes and commit message for a specific version

```
1 git show HEAD~2 Malaria_incidence_analysis.py
```

Exploring History (Continued)

Referencing Commits

- Every commit has a unique 40-character identifier (hash)
- Shortened hashes (first 7–8 characters) can be used

```
1 git diff f22b25e Malaria_incidence_analysis.py
```

Recover File Versions - `git restore`

Restores file contents from a specified source without affecting commit history.

Discard unstaged changes

```
1 git restore Malaria_incidence_analysis.py
```

Unstage a file

```
1 git restore --staged Malaria_incidence_analysis.py
```

Restore from a specific commit

```
1 git restore --source f22b25e Malaria incidence analysis.py
```

Exploring History (Continued)

Move Branch Pointer - `git reset`

Moves the current branch pointer to a specified commit.

Unstage a file

```
1 git reset HEAD Malaria_incidence_analysis.py
```

Undo last commit, preserve changes

```
1 git reset --soft HEAD~1
```

Undo last commit, unstage changes

```
1 git reset HEAD~1
```

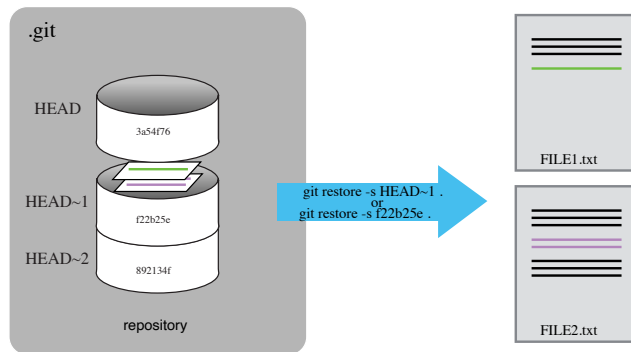
Undo last commit, discard everything

```
1 git reset --hard HEAD~1
```

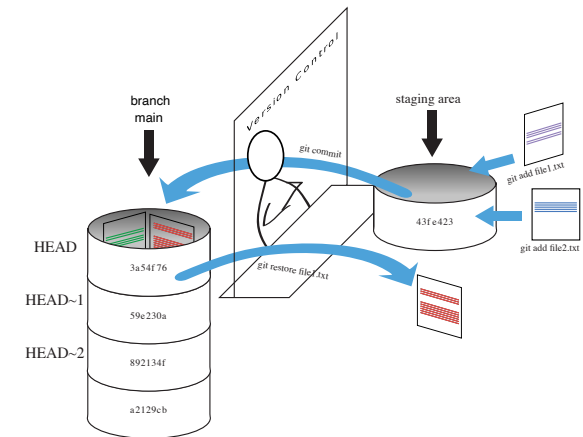
Command Reference

Situation	Command
Discard unsaved changes in a file	<code>git restore file</code>
Accidentally staged a file	<code>git restore --staged file</code>
Retrieve an older version of a file	<code>git restore --source commit file</code>
Undo commit but keep work in progress	<code>git reset --soft HEAD~1</code>
Undo commit and work on changes fresh	<code>git reset HEAD~1</code>
Permanently erase last commit	<code>git reset --hard HEAD~1</code>

Git Workflow Visualization



Git Restore



Git Workflow

The fact that files can be reverted one by one tends to change the way people organize their work.

Challenge: Recovering Older Versions

Anwar has made changes to the Python script he has been working on for weeks analyzing malaria incidence data across Ethiopian zones. The modifications he made this morning “broke” the script and it no longer runs. He has spent ~1hr trying to fix it, with no luck.

Luckily, he has been keeping track of his project’s versions using Git. Which commands below will let him recover the last committed version of his Python script called `malaria_incidence_analysis.py`?

- `git restore`
- `git restore malaria_incidence_analysis.py`
- `git restore -s HEAD~1 malaria_incidence_analysis.py`
- `git restore -s <unique ID of last commit> malaria_incidence_analysis.py`
- Both 2 and 4

► View Solution

Challenge: Reverting a Commit

Anwar is collaborating with colleagues at the Ethiopian Public Health Institute on his malaria incidence analysis script. He discovers his most recent commit contains an error in the data cleaning function and to undo it. The correction must be properly recorded so all team members receive the fix.

The command `git revert [commit ID]` creates a new commit that reverses the changes from a specific commit.

What is the missing command in Anwar's workflow?

1. _____ – View project history to locate the erroneous commit ID
2. Copy the commit ID (first few characters, e.g., 0b1d055)
3. `git revert [commit ID]`
4. Enter the new commit message
5. Save and close

► View Solution

Challenge: Understanding Workflow and History

What is the output of the last command in this sequence?

```
1 cd Malaria
2 echo "def load_malaria_data(): pass" > eda/malaria_incidence_analysis.py
3 git add eda/malaria_incidence_analysis.py
4 echo "def calculate_incidence_rate(): pass" >> eda/malaria_incidence_analysis.py
5 git commit -m "Initial malaria analysis setup"
6 git restore eda/malaria_incidence_analysis.py
7 cat eda/malaria_incidence_analysis.py
```

1. `def calculate_incidence_rate(): pass`
2. `def load_malaria_data(): pass`
3. Both functions
4. Error

► [View Solution](#)

Ignoring Things

Why Ignore Files?

Git can track every file in a project, but some files should not be version controlled:

- Editor backup files (e.g., *.swp, *~)
- Intermediate data files or outputs
- System files (e.g., .DS_Store)
- Sensitive information (e.g., API keys, passwords)
- Large binary files

Creating Dummy Files

Let's create some dummy files to demonstrate:

```
1 mkdir figures
2 touch map1.png map2.png map3.png figures/ethiopia_map.jpg figures/tigray_map.jpg
```

Check the status:

```
1 git status
```

```
1 On branch main
2 Untracked files:
3   (use "git add <file>..." to include in what will be committed)
4
5     map1.png
6     map2.png
7     map3.png
8     figures/
9
10 nothing added to commit but untracked files present (use "git add" to track)
```

Creating `.gitignore`

Create a file called `.gitignore` in the root of your project:

```
1 nano .gitignore
```

Add patterns for files and directories to ignore:

```
*.png  
figures/
```

Save and exit.

Verifying `.gitignore`

Check the contents:

```
1 cat .gitignore
```

```
1 *.png  
2 figures/
```

How `.gitignore` Works

- *.png – ignores all files with the .png extension
- figures/ – ignores the entire figures directory and its contents

Important: These patterns only affect untracked files. Files already tracked by Git will continue to be tracked regardless of .gitignore rules.

Cleaner Status

After adding `.gitignore`, `git status` becomes cleaner:

```
1 git status
```

```
1 On branch main
2 Untracked files:
3   (use "git add <file>..." to include in what will be committed)
4
5   .gitignore
6
7 nothing added to commit but untracked files present (use "git add" to track)
```

Committing `.gitignore`

The `.gitignore` file itself should be tracked so all collaborators ignore the same files:

```
1 git add .gitignore
2 git commit -m "Ignore png files and the figures folder"
3 git status
```

```
1 On branch main
2 nothing to commit, working tree clean
```

Attempting to Add Ignored Files

Git prevents accidental addition of ignored files:

```
1 git add map1.png
```

```
1 The following paths are ignored by one of your .gitignore files:
2 map1.png
3 Use -f if you really want to add them.
```

Force Adding Ignored Files

If you really need to track an ignored file, use `-f` flag:

```
1 git add -f map1.png
```

Viewing Ignored Files

See all ignored files in your status:

```
1 git status --ignored
```

```
1 On branch main
2 Ignored files:
3   (use "git add -f <file>..." to include in what will be committed)
4
5     map1.png
6     map2.png
7     map3.png
8     figures/
9
10 nothing to commit, working tree clean
```

Common `.gitignore` Patterns

Pattern	Matches
<code>*.log</code>	All files ending in <code>.log</code>
<code>temp/</code>	The entire <code>temp</code> directory
<code>*.csv</code>	All CSV files
<code>!important.csv</code>	Exception: track <code>important.csv</code> even if <code>.csv</code> is ignored
<code>data/raw/</code>	The <code>data/raw</code> directory
<code>*.pyc</code>	Compiled Python files
<code>.DS_Store</code>	macOS system files

Challenge: Ignoring Nested Files

Given a directory structure:

```
data/raw/survey/  
data/raw/interviews/
```

How would you ignore only `data/raw/survey` while continuing to track `data/raw/interviews`?

► [View Solution](#)

Challenge: Including Specific Files

How would you ignore all `.png` files in your root directory except for `ethiopia_boundary.png`? Hint: Find out what `!` (the exclamation point operator) does

► View Solution

Challenge: Ignoring Nested Files Variation

Given a directory structure:

```
1 data/raw/Malaria/  
2 data/raw/malaria/  
3 data/raw/hiv/  
4 data/raw/mch/
```

How would you ignore all of the contents in the data/raw folder, but not `data/raw/hiv`?

► View Solution

Challenge: Ignoring All Data Files in a Directory

Assuming you have an empty `.gitignore` file, and given a directory structure that looks like:

```
1 data/incidence/ethiopia/Malaria/Malaria_2020.csv
2 data/incidence/ethiopia/Malaria/Malaria_2021.csv
3 data/incidence/ethiopia/Malaria/Malaria_2022.csv
4 data/incidence/ethiopia/Malaria/README.txt
5 data/incidence/plots/
```

What's the shortest `.gitignore` rule you could write to ignore all `.csv` files in `data/incidence/ethiopia/Malaria/`? Do not ignore the `README.txt`.

► View Solution

Challenge: Ignoring All Data Files in the Repository

Let us assume you have many `.csv` files in different subdirectories of your repository. For example, you have:

```
1 results/Malaria_incidence.csv
2 data/malaria/2020/cases.csv
3 data/malaria/2021/cases.csv
4 data/hiv/regional/art_coverage.csv
```

How do you ignore all the `.csv` files, without explicitly listing the names of the corresponding folders?

► [View Solution](#)

Challenge: The Order of Rules

Given a `.gitignore` file with the following contents:

```
*.CSV  
!*.CSV
```

What will be the result?

► [View Solution](#)

Challenge: Log Files

You wrote a script that creates many intermediate log files of the form `log_01`, `log_02`, `log_03`, etc. for training model runs. You want to keep them but you do not want to track them through Git.

Part 1: Create the Ignore Pattern

Write **one** `.gitignore` entry that excludes files of the form `log_01`, `log_02`, etc.

► [View Solution](#)

Challenge: Log Files (Continued)

Part 2: Test the Pattern

Create dummy files and verify they are ignored:

```
1 touch log_01 log_02 log_03 log_04
2 git status --ignored
```

► View Solution

Part 3: Track an Important File

You find that `log_01` is very important after all. Add it to tracked files without changing `.gitignore`.

► View Solution

Challenge: Log Files (Continued)

Discuss Common Files to Ignore

What other types of files might you want to exclude via `.gitignore`?

► [View Solution](#)

Remotes in GitHub

Why Remote Repositories?

- Version control really comes into its own when we begin to collaborate with other people. We already have most of the machinery we need to do this; the only thing missing is to copy changes from one repository to another.
- Systems like Git allow us to move work between any two repositories. In practice, though, it's easiest use one copy as a central hub, and to keep it on the web rather than on someone's laptop. Most programmers use hosting services like GitHub, BitBucket or GitLab to hold those main copies.
- Let's start by sharing the changes we've made to our current project with the world. To this end we are going to create a *remote* repository that will be linked to our *local* repository.

1. Create a Remote Repository

Log in to GitHub, then click on the icon in the top right corner to create a new repository called [Malaria](#)



Creating a repository on GitHub

Name your repository “Malaria” and then click “Create Repository”.

Note: Since this repository will be connected to a local repository, it needs to be empty. Leave “Initialize repository with a README” unchecked, and keep “None” as options for both “Add .gitignore” and “Add license.”

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk ().*

Repository template

No template ▾

Start your repository with a template repository's contents.

Owner *

 alflin ▾

Repository name *

/ recipes

✔ recipes is available.

Great repository names are short and memorable. Need inspiration? How about **laughing-umbrella** ?

Description (optional)

- ☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

- ☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

License: None ▾

A license tells others what they can and can't do with your code. [Learn more about licenses](#).

 You are creating a public repository in your personal account.

Repository Created

As soon as the repository is created, GitHub displays a page with a URL and some information on how to configure your local repository:

 recipes Public

 Pin

 Unwatch 1

 Fork 0

 Star 0



Start coding with Codespaces

Add a README file and start coding in a secure, configurable, and dedicated development environment.

Create a codespace



Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

HTTPS

SSH

https://github.com/ alflin /recipes.git



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# recipes" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ alflin /recipes.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin https://github.com/ alflin /recipes.git
git branch -M main
git push -u origin main
```

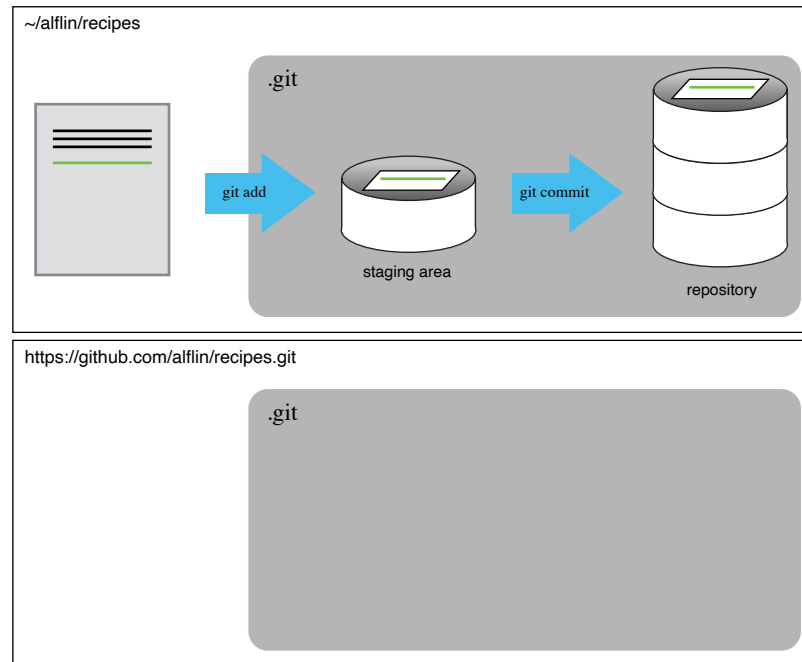


Repository summary page

This effectively does the following on GitHub's servers:

```
1 mkdir malaria_test
2 cd malaria_test
3 git init
```

Now that we have two repositories, we need a diagram like this:

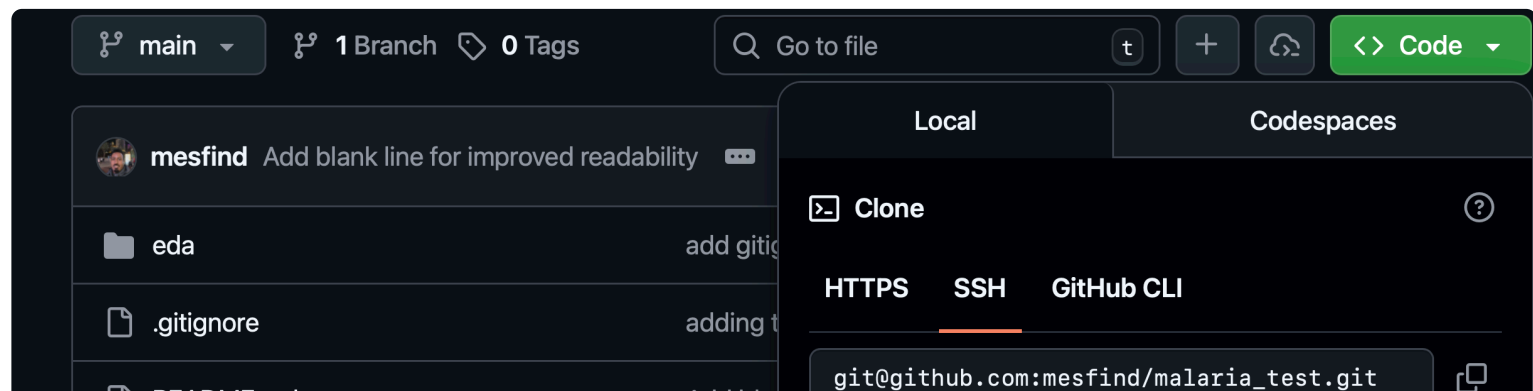


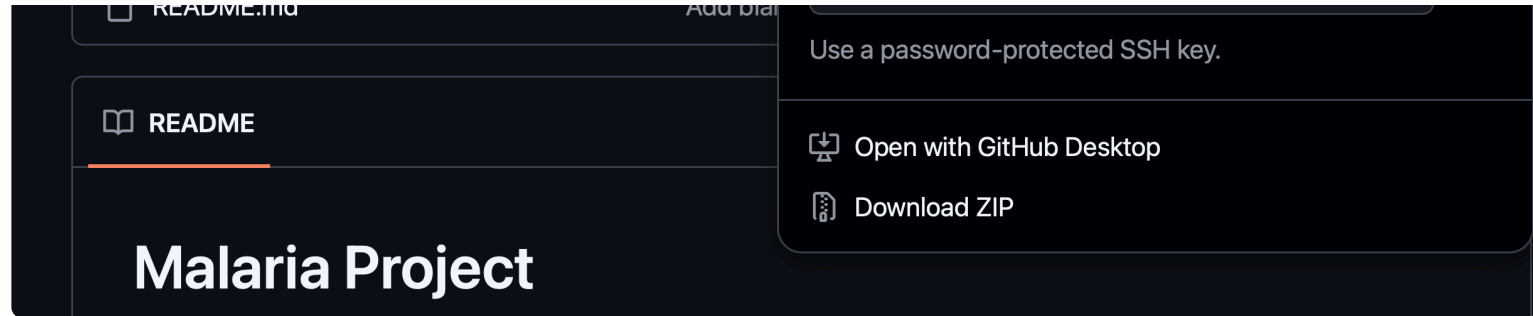
Freshly made GitHub repo

Note that our local repository still contains our earlier work, but the remote repository on GitHub appears empty as it doesn't contain any files yet.

2. Connect Local to Remote Repository

Now we connect the two repositories. We do this by making the GitHub repository a remote for the local repository. The home page of the repository on GitHub includes the URL string we need to identify it:



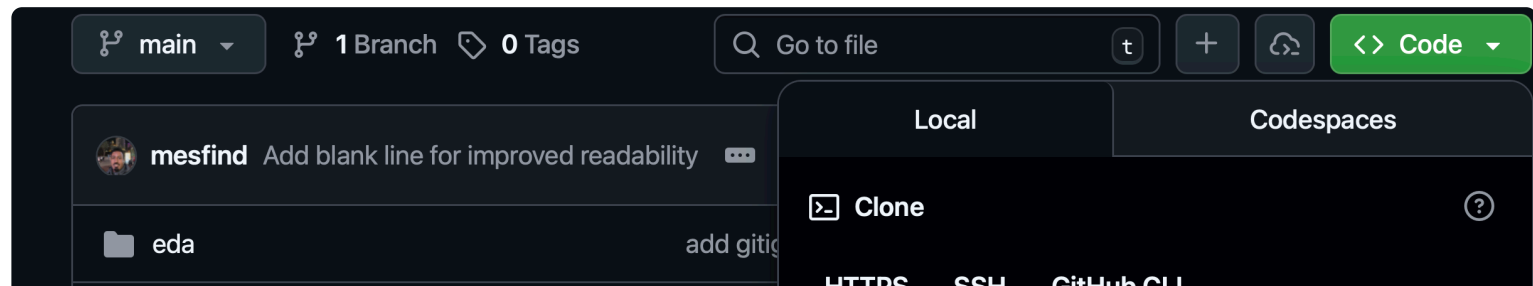


Repository URL

Click on the 'SSH' link to change the protocol from HTTPS to SSH.

HTTPS vs. SSH

We use SSH here because, while it requires some additional configuration, it is a security protocol widely used by many applications.



Adding the Remote

Copy that URL from the browser, go into the local `malaria` repository, and run this command:

```
1 git remote add origin git@github.com:mesfind/malaria_test.git
```

Make sure to use the URL for your repository rather than Anwar's: the only difference should be your username.

`origin` is a local name used to refer to the remote repository. It could be called anything, but `origin` is a convention that is often used by default in git and GitHub.

We can check that the command has worked by running `git remote -v`:

```
1 git remote -v
```

```
1 origin  git@github.com:anwartaju/malaria_test.git (fetch)
2 origin  git@github.com:anwartaju/malaria_test.git (push)
```

SSH Background and Setup

Before Anwar can connect to a remote repository, he needs to set up a way for his computer to authenticate with GitHub so it knows it's him trying to connect.

We are going to set up the method that is commonly used by many different services to authenticate at the command line. This method is called Secure Shell Protocol (SSH). SSH uses what is called a key pair: a public key and a private key.

You can think of the public key as a padlock, and only you have the key (the private key) to open it. You give this padlock to GitHub and say "lock the communications to my account with this so that only someone who has my private key can unlock communications."

Keeping your keys secure

You shouldn't really forget about your SSH keys, since they keep your account secure. It's good practice to check your SSH keys every so often to ensure they are still secure, up to date, and that there are no unauthorized keys.

Checking for Existing SSH Keys

We will run the list command to check what key pairs already exist on your computer:

```
1 ls -al ~/.ssh
```

If SSH has been set up, the public and private key pairs will be listed. The file names are either `id_ed25519/id_ed25519.pub` or `id_rsa/id_rsa.pub` depending on how the key pairs were set up. If they don't exist, we need to create them.

Create an SSH Key Pair

To create an SSH key pair, use this command:

```
1 ssh-keygen -t ed25519 -C "anwar.taju@ephi.gov.et"
```

If you are using a legacy system that doesn't support the Ed25519 algorithm, use: `$ ssh-keygen -t rsa 4096 -C "your_email@example.com"`

```
1 Generating public/private ed25519 key pair.  
2 Enter file in which to save the key (/Users/Anwar/.ssh/id_ed25519):
```

Press Enter to use the default file.

```
1 Created directory '/Users/Anwar/.ssh'.  
2 Enter passphrase (empty for no passphrase):
```

Now, it is prompting for a passphrase. Be sure to use something memorable or save your passphrase somewhere. Note that, when typing a passphrase on a terminal, there won't be any visual feedback.

SSH Key Created

After entering the same passphrase a second time, we receive the confirmation:

```
1 Your identification has been saved in /Users/Anwar/.ssh/id_ed25519
2 Your public key has been saved in /Users/Anwar/.ssh/id_ed25519.pub
3 The key fingerprint is:
4 SHA256:SMSPIStNyA00KPXuYu94KpZgRAYjgt9g4BA4kFy3g1o anwar.taju@ephi.gov.et
```

Now that we have generated the SSH keys, we will find the SSH files when we check:

```
1 ls -al ~/.ssh
```

```
1 drwxr-xr-x 1 Anwar  197121  0 Jul 16 14:48 ./
2 drwxr-xr-x 1 Anwar  197121  0 Jul 16 14:48 ../
3 -rw-r--r-- 1 Anwar  197121 419 Jul 16 14:48 id_ed25519
4 -rw-r--r-- 1 Anwar  197121 106 Jul 16 14:48 id_ed25519.pub
```

Copy the Public Key to GitHub

First, we need to copy the public key. Be sure to include the `.pub` at the end:

```
1 cat ~/.ssh/id_ed25519.pub
```

```
1 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDmRA3d51X0uu9wXek559gfn6UFNF69yZjChyBIU2qKI anwar.taju@ephi.gov.et
```

Now, going to GitHub.com, click on your profile icon in the top right corner to get the drop-down menu “Settings”, then on the settings page, click “SSH and GPG keys” on the left side. Click the “New SSH key” button. Add a title (e.g., “EPHI Workstation”), paste your SSH key into the field, and click “Add SSH key”.

Testing SSH Authentication

Now that we've set that up, let's check our authentication:

```
1 ssh -T git@github.com
```

```
1 Hi Anwar Taju! You've successfully authenticated, but GitHub does not provide shell access.
```

Good! This output confirms that the SSH key works as intended. We are now ready to push our work to remote repository.

Push Local Changes to a Remote

Now that authentication is setup, we can return to the remote. This command will push the changes from local repository to the repository on GitHub:

```
1 git push origin main
```

```
1 Enumerating objects: 16, done.
2 Counting objects: 100% (16/16), done.
3 Delta compression using up to 8 threads.
4 Compressing objects: 100% (11/11), done.
5 Writing objects: 100% (16/16), 1.45 KiB | 372.00 KiB/s, done.
6 Total 16 (delta 2), reused 0 (delta 0)
7 remote: Resolving deltas: 100% (2/2), done.
8 To https://github.com/anwartaju/Malaria.git
9 * [new branch]      main -> main
```

Proxy Settings

Proxy Configuration

If the network you are connected to uses a proxy, your command may fail with “Could not resolve hostname”. To fix this issue, you need to tell Git about the proxy:

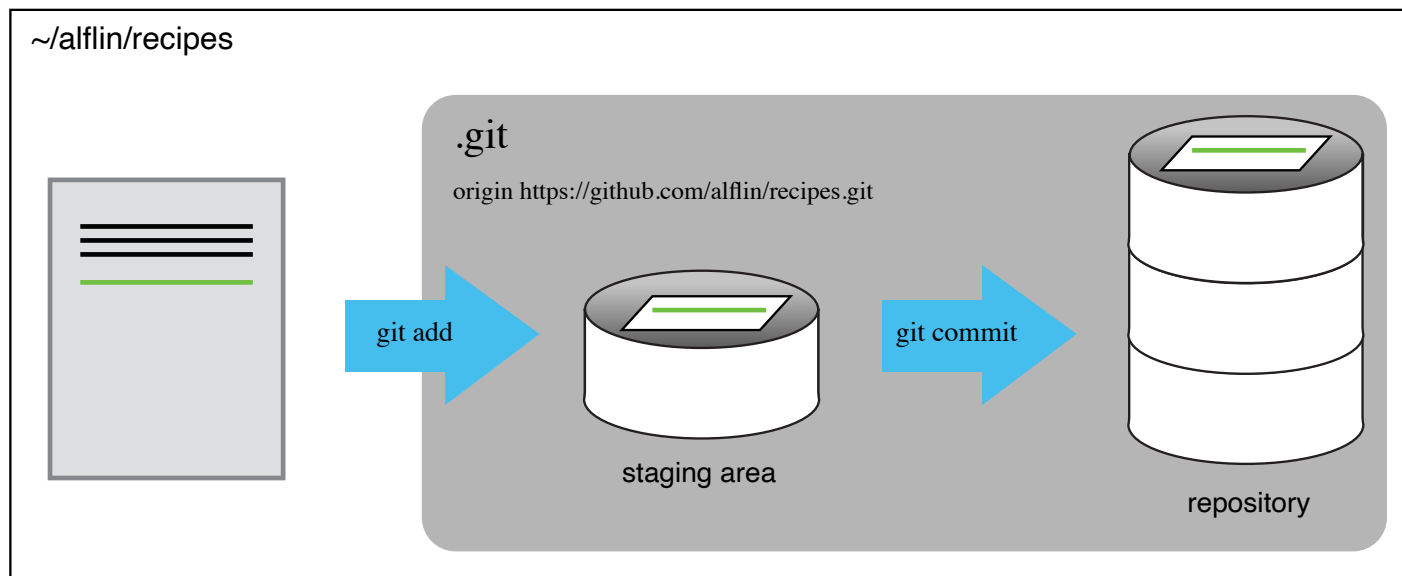
```
1 git config --global http.proxy http://user:password@proxy.url
2 git config --global https.proxy https://user:password@proxy.url
```

When you connect to another network that doesn't use a proxy, you will need to disable the proxy:

```
1 git config --global --unset http.proxy
2 git config --global --unset https.proxy
```

Repository State After Push

Our local and remote repositories are now in this state:



After first push

Challenge: GitHub GUI

Browse to your [Malaria](#) repository on GitHub. Under the Code tab, find and click on the text that says “commits”. Hover over, and click on, the three buttons to the right of each commit. What information can gather from these buttons? How would you get that same information in the shell?

► View Solution

Challenge: Push vs. Commit

How is “git push” different from “git commit”?

► [View Solution](#)

Challenge: GitHub License and README Files

In this episode we learned about creating a remote repository on GitHub, but when you initialized your repo, you didn't add a README.md or a license file. If you had, what do you think would have happen when you tried to link your local and remote repositories?

► [View Solution](#)

Collaborating

Questions

- How can I use version control to collaborate with other people?

Setting Up Collaboration

For the next step, get into pairs. One person will be the “Owner” and the other will be the “Collaborator”. The goal is that the Collaborator add changes into the Owner’s repository. We will switch roles at the end, so each person will play Owner and Collaborator.

Practicing By Yourself

If you’re working through this lesson on your own, you can carry on by opening a second terminal window. This window will represent your partner, working on another computer. You won’t need to give anyone access on GitHub because both ‘partners’ are you.

Adding a Collaborator

The Owner needs to give the Collaborator access. In your repository page on GitHub, click the “Settings” button on the right, select “Collaborators”, click “Add people”, and then enter your partner’s username.

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Code security and analysis

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Who has access

PUBLIC REPOSITORY



This repository is public and visible to anyone.

[Manage](#)

DIRECT ACCESS



0 collaborators have access to this repository. Only you can contribute to this repository.

Manage access



You haven't invited any collaborators yet

Add people

Cloning a Repository

Next, the Collaborator needs to download a copy of the Owner's repository to her machine. This is called "cloning a repo".

The Collaborator doesn't want to overwrite her own version of [Malaria](#), so needs to clone the Owner's repository to a different location.

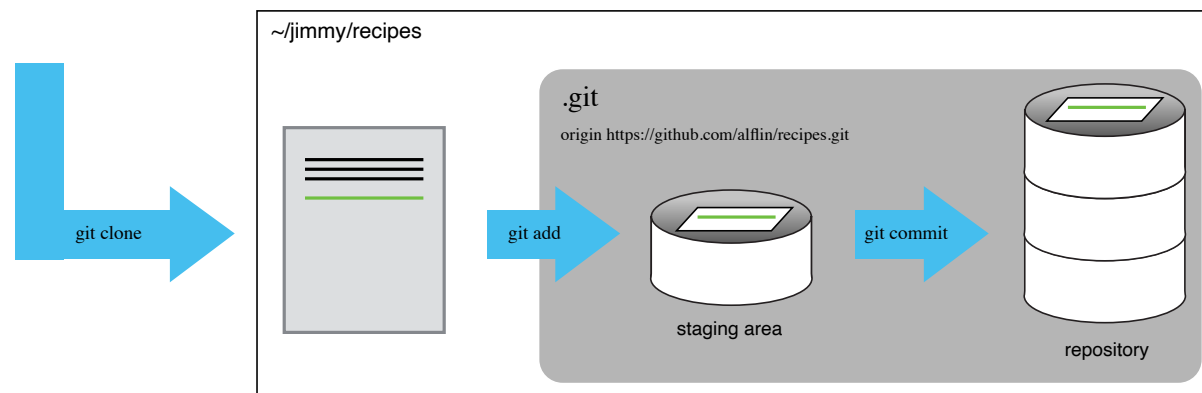
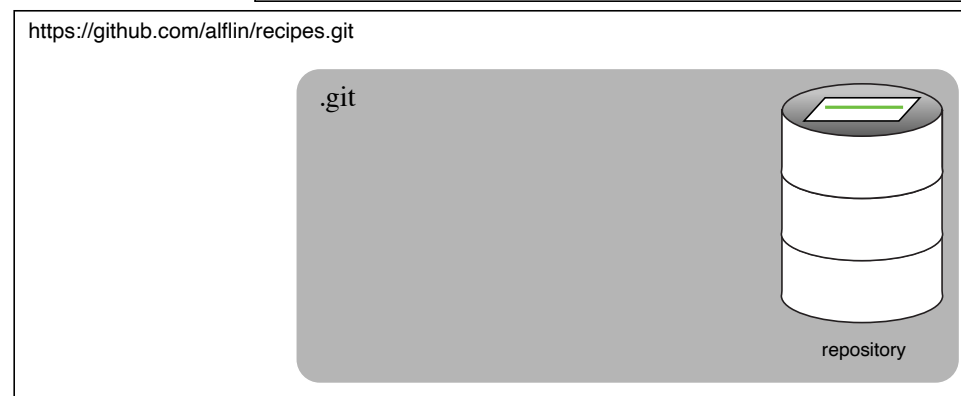
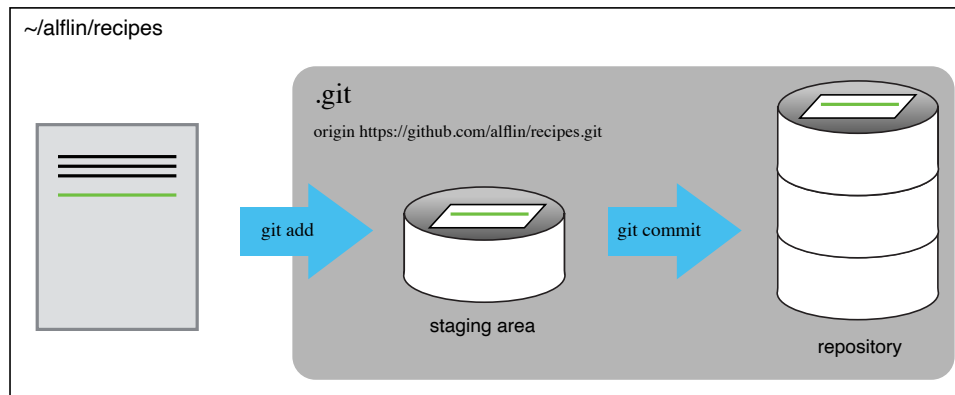
To clone the Owner's repo into her [Desktop](#) folder, the Collaborator enters:

```
1 git clone git@github.com:anwartaju/Malaria.git ~/Desktop/anwar-health-data
```

Replace 'anwartaju' with the Owner's username.

If you choose to clone without the clone path specified at the end, you will clone inside your own folder sure to navigate to the [Desktop](#) folder first.

Cloning Visualization



Git Clone

Pushing Changes

```
1 git add malaria_analysis.py
2 git commit -m "Add malaria incidence calculation function"
```

```
1 1 file changed, 8 insertion(+)
2 create mode 100644 malaria_analysis.py
```

Then push the change to the *Owner's repository* on GitHub:

```
1 git push origin main
```

```
1 Enumerating objects: 4, done.
2 Counting objects: 4, done.
3 Delta compression using up to 4 threads.
4 Compressing objects: 100% (2/2), done.
5 Writing objects: 100% (3/3), 306 bytes, done.
6 Total 3 (delta 0), reused 0 (delta 0)
7 To https://github.com/anwartaju/Malaria.git
8 9272da5..29aba7c main -> main
```

Note that we didn't have to create a remote called `origin`: Git uses this name by default when we clone repository.

Viewing Changes

Take a look at the Owner's repository on GitHub again, and you should be able to see the new commit from the Collaborator. You may need to refresh your browser to see the new commit.

Some more about remotes

In this episode and the previous one, our local repository has had a single “remote”, called `origin`. A remote is a copy of the repository that is hosted somewhere else, that we can push to and pull from, and there's no reason that you have to work with only one.

- `git remote -v` lists all the remotes that are configured
- `git remote add [name] [url]` is used to add a new remote
- `git remote remove [name]` removes a remote
- `git remote set-url [name] [newurl]` changes the URL associated with the remote
- `git remote rename [oldname] [newname]` changes the local alias

Pulling Changes

To download the Collaborator's changes from GitHub, the Owner now enters:

```
1 git pull origin main
```

```
1 remote: Enumerating objects: 4, done.
2 remote: Counting objects: 100% (4/4), done.
3 remote: Compressing objects: 100% (2/2), done.
4 remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
5 Unpacking objects: 100% (3/3), done.
6 From https://github.com/anwartaju/Malaria
7  * branch          main      -> FETCH_HEAD
8   9272da5..29aba7c  main      -> origin/main
9 Updating 9272da5..29aba7c
10 Fast-forward
11  malaria_analysis.py | 5 +
12  1 file changed, 5 insertion(+)
13 create mode 100644 malaria_analysis.py
```

Now the three repositories (Owner's local, Collaborator's local, and Owner's on GitHub) are back in syn

Basic Collaborative Workflow

A Basic Collaborative Workflow

In practice, it is good to be sure that you have an updated version of the repository you are collaborating on, so you should `git pull` before making our changes. The basic collaborative workflow would be:

1. Update your local repo with `git pull origin main`
2. Make your changes and stage them with `git add`
3. Commit your changes with `git commit -m`
4. Upload the changes to GitHub with `git push origin main`

It is better to make many commits with smaller changes rather than one commit with massive changes: small commits are easier to read and review.

Challenge: Switch Roles and Repeat

Switch roles and repeat the whole process. The new Owner should give access to the new Collaborator, will clone the repository, make a change, push it, and the Owner will pull it.

Challenge: Review Changes

The Owner pushed commits to the repository without giving any information to the Collaborator. How Collaborator find out what has changed with command line? And on GitHub?

► [View Solution](#)

Challenge: Comment Changes in GitHub

The Collaborator has some questions about one line change made by the Owner and has some suggestions to propose. For example, the Owner might have added a function to calculate Malaria incidence but the Collaborator notices a potential error in the formula.

With GitHub, it is possible to comment on the diff of a commit. Over the line of code to comment, a blue comment icon appears to open a comment window.

The Collaborator posts her comments and suggestions using the GitHub interface.

► [View Solution](#)

Challenge: Version History, Backup, and Version Control

Some backup software can keep a history of the versions of your files. They also allow you to recover specific versions. How is this functionality different from version control? What are some of the benefits of using version control, Git and GitHub?

► [View Solution](#)

Conflicts

Questions

- What do I do when my changes conflict with someone else's?

Understanding Conflicts

As soon as people can work in parallel, they'll likely step on each other's toes. This will even happen with a single person: if we are working on analysis code on both our laptop and a server, we could make different changes to each copy. Version control helps us manage these conflicts by giving us tools to resolve overlapping changes.

To see how we can resolve conflicts, we must first create one. The file `Malaria_incidence_analysis.py` currently looks like this in both partners' copies of our `Malaria` repository:

```
1 cat Malaria_incidence_analysis.py
```

```
1 # Malaria Incidence Analysis - Ethiopia
2
3 def load_Malaria_data(filepath):
4     data = pd.read_csv(filepath)
5     return data
6
7 def calculate_incidence(cases, population):
8     incidence = (cases / population) * 100000
9     return incidence
```

Creating a Conflict - Collaborator's Change

Let's add a line to the collaborator's copy only:

```
1 nano Malaria_incidence_analysis.py
2 cat Malaria_incidence_analysis.py

1 # Malaria Incidence Analysis - Ethiopia
2
3 def load_Malaria_data(filepath):
4     data = pd.read_csv(filepath)
5     return data
6
7 def calculate_incidence(cases, population):
8     incidence = (cases / population) * 100000
9     return incidence
10
11 # Add function to visualize Malaria trends
12 def plot_Malaria_trends(data):
13     import matplotlib.pyplot as plt
14     plt.plot(data['year'], data['incidence'])
15     plt.title('Malaria Incidence Trends in Ethiopia')
16     plt.show()
```

and then push the change to GitHub:

```
1 git add Malaria_incidence_analysis.py
2 git commit -m "Add Malaria visualization function"
3 git push origin main
```

Creating a Conflict - Owner's Change

Now let's have the owner make a different change to their copy *without* updating from GitHub:

```
1 nano Malaria_incidence_analysis.py
2 cat Malaria_incidence_analysis.py

1 # Malaria Incidence Analysis - Ethiopia
2
3 def load_Malaria_data(filepath):
4     data = pd.read_csv(filepath)
5     return data
6
7 def calculate_incidence(cases, population):
8     incidence = (cases / population) * 100000
9     return incidence
10
11 # Add function to calculate age-standardized rates
12 def age_standardize_incidence(data, standard_population):
13     age_adj_incidence = data['incidence'] * standard_population
14     return age_adj_incidence
```

We can commit the change locally:

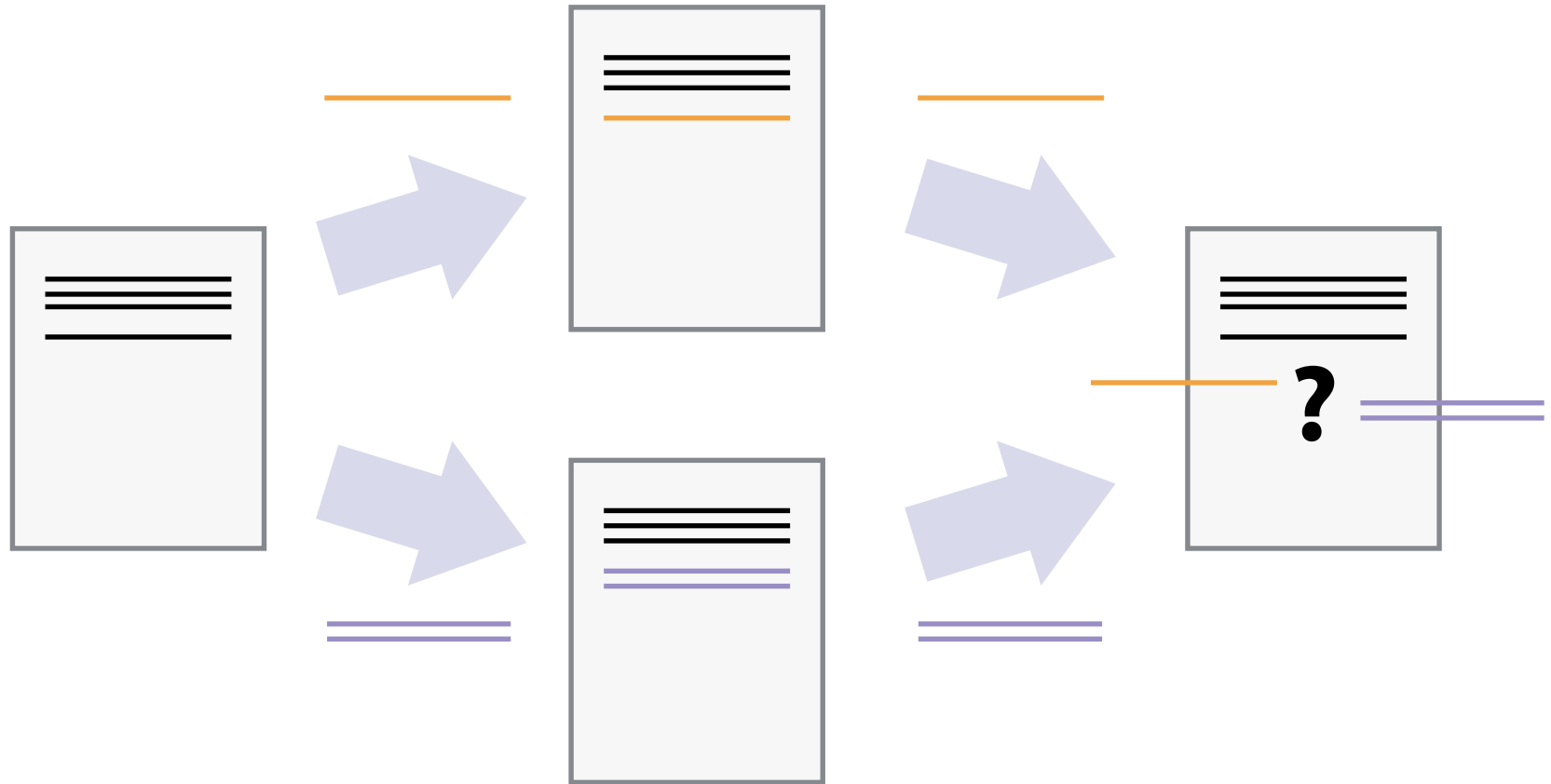
```
1 git add Malaria_incidence_analysis.py
2 git commit -m "Add age-standardization function"
```

Push Rejected

But Git won't let us push it to GitHub:

```
1 git push origin main
```

```
1 To https://github.com/anwartaju/Malaria.git
2 ! [rejected]          main -> main (fetch first)
3 error: failed to push some refs to 'https://github.com/anwartaju/Malaria.git'
4 hint: Updates were rejected because the remote contains work that you do
5 hint: not have locally. This is usually caused by another repository pushing
6 hint: to the same ref. You may want to first integrate the remote changes
7 hint: (e.g., 'git pull ...') before pushing again.
```



Conflict Diagram

Git rejects the push because it detects that the remote repository has new updates that have not been incorporated into the local branch.

Pulling Changes

What we have to do is pull the changes from GitHub, merge them into the copy we're currently working and then push that. Let's start by pulling:

```
1 git pull origin main
```

```
1 remote: Enumerating objects: 5, done.
2 remote: Counting objects: 100% (5/5), done.
3 remote: Compressing objects: 100% (1/1), done.
4 remote: Total 3 (delta 2), reused 3 (delta 2), pack-reused 0
5 Unpacking objects: 100% (3/3), done.
6 From https://github.com/anwartaju/Malaria
7  * branch          main      -> FETCH_HEAD
8   29aba7c..dabb4c8  main      -> origin/main
9 Auto-merging Malaria_incidence_analysis.py
10 CONFLICT (content): Merge conflict in Malaria_incidence_analysis.py
11 Automatic merge failed; fix conflicts and then commit the result.
```

Configuring Pull Behavior

You may need to tell Git what to do

If you see output about divergent branches, Git is asking what it should do. In newer versions of Git it gives you option of specifying different behaviours when a pull would merge divergent branches. In our case we want 'the default strategy'. To use this strategy run:

```
1 git config pull.rebase false
```

Then attempt the pull again:

```
1 git pull origin main
```

Viewing the Conflict

The `git pull` command updates the local repository to include those changes already included in the remote repository. After the changes from remote branch have been fetched, Git detects that changes made to the local copy overlap with those made to the remote repository, and therefore refuses to merge the two versions. A conflict is marked in the affected file:

```
1 cat Malaria_incidence_analysis.py
```

```
1 # Malaria Incidence Analysis - Ethiopia
2
3 def load_Malaria_data(filepath):
4     data = pd.read_csv(filepath)
5     return data
6
7 def calculate_incidence(cases, population):
8     incidence = (cases / population) * 100000
9     return incidence
10
11 <<<<<< HEAD
12 # Add function to calculate age-standardized rates
13 def age_standardize_incidence(data, standard_population):
14     age_adj_incidence = data['incidence'] * standard_population
15     return age_adj_incidence
16 =====
17 # Add function to visualize Malaria trends
```

Understanding Conflict Markers

Our change is preceded by <<<<<< HEAD. Git has then inserted ===== as a separator between the changes and marked the end of the content downloaded from GitHub with >>>>>>. (The string of letters and digits after that marker identifies the commit we've just downloaded.)

It is now up to us to edit this file to remove these markers and reconcile the changes. We can do anything we want: keep the change made in the local repository, keep the change made in the remote repository, write something new to replace both, or get rid of the change entirely.

Let's replace both so that the file looks like this:

```
1 cat Malaria_incidence_analysis.py
```

```
1 # Malaria Incidence Analysis - Ethiopia
2
3 def load_Malaria_data(filepath):
4     data = pd.read_csv(filepath)
5     return data
6
7 def calculate_incidence(cases, population):
8     incidence = (cases / population) * 100000
9     return incidence
10
11 # Add function to calculate age-standardized rates
12 def age_standardize_incidence(data, standard_population):
13     age_adj_incidence = data['incidence'] * standard_population
14     return age_adj_incidence
15
16 # Add function to visualize Malaria trends
17 def plot_Malaria_trends(data):
18     import matplotlib.pyplot as plt
19     plt.plot(data['year'], data['incidence'])
20     plt.title('Malaria Incidence Trends in Ethiopia')
21     plt.show()
```

Completing the Merge

To finish merging, we add `Malaria_incidence_analysis.py` to the changes being made by the merge a commit:

```
1 git add Malaria_incidence_analysis.py
2 git status
```

```
1 On branch main
2 All conflicts fixed but you are still merging.
3   (use "git commit" to conclude merge)
4
5 Changes to be committed:
6
7   modified:   Malaria_incidence_analysis.py
```

```
1 git commit -m "Merge changes from GitHub - combine age standardization and visualization"
```

```
1 [main 2abf2b1] Merge changes from GitHub - combine age standardization and visualization
```

Now we can push our changes to GitHub:

```
1 git push origin main
```

After the Merge

Git keeps track of what we've merged with what, so we don't have to fix things by hand again when the collaborator who made the first change pulls again:

```
1 git pull origin main

1 remote: Enumerating objects: 10, done.
2 remote: Counting objects: 100% (10/10), done.
3 remote: Compressing objects: 100% (2/2), done.
4 remote: Total 6 (delta 4), reused 6 (delta 4), pack-reused 0
5 Unpacking objects: 100% (6/6), done.
6 From https://github.com/anwartaju/Malaria
7  * branch          main      -> FETCH_HEAD
8   dabb4c8..2abf2b1  main      -> origin/main
9 Updating dabb4c8..2abf2b1
10 Fast-forward
11  Malaria_incidence_analysis.py | 2 +-
12  1 file changed, 1 insertion(+), 1 deletion(-)
```

We get the merged file:

```
1 cat Malaria_incidence_analysis.py
```

Both functions are now present in the file.

Strategies to Minimize Conflicts

Git's ability to resolve conflicts is very useful, but conflict resolution costs time and effort, and can introduce errors if conflicts are not resolved correctly. If you find yourself resolving a lot of conflicts in a project, consider these approaches:

Technical approaches: - Pull from upstream more frequently, especially before starting new work - Use feature branches to segregate work, merging to main when complete - Make smaller, more atomic commits - Push work when it is done - Break large files into smaller ones

Project management strategies: - Clarify who is responsible for what areas - Discuss task order with collaborators - Establish project conventions for code style

Challenge: Solving Conflicts

Clone the repository created by your instructor. Add a new file to it, and modify an existing file (your instructor will tell you which one). When asked by your instructor, pull changes from the repository to create a conflict, then resolve it.

Challenge: Conflicts on Non-textual Files

What does Git do when there is a conflict in an image (like a map of Malaria incidence in Ethiopia) or so other non-textual file that is stored in version control?

► [View Solution](#)

Challenge: A Typical Work Session

You sit down at your computer to work on a shared project that is tracked in a remote Git repository. During your work session, you take the following actions, but not in this order:

- *Make changes* by appending the number `100` to a text file `cases.csv`
- *Update remote* repository to match the local repository
- *Celebrate* your success with some Ethiopian coffee
- *Update local* repository to match the remote repository
- *Stage changes* to be committed
- *Commit changes* to the local repository

In what order should you perform these actions to minimize the chances of conflicts?

► [View Solution](#)

Advanced Branching for DS Experiments

In Data Science, branching isn't just for features—it's for tracking experiments. Git Flow for ML enables baseline tracking, hyperparameter tuning, and ensemble development.

```
1 # Setup a mock repo structure for a climate impact study
2 mkdir -p ds_project/{data,notebooks,models}
3 cd ds_project
4 git init
5
6 # 1. Main baseline (simple linear regression on climate data)
7 git checkout -b main
8 echo "# Baseline: Temp vs Yield (Linear)" > notebooks/baseline.py
9 git add . && git commit -m "feat: baseline linear model"
10
```

Advanced Branching for DS Experiments (Continued)



Tip

Git Flow for DS: Keep `main` as production/stable metrics. Use `experiment/` prefixes for hyperparameter runs and `feat/` for new data sources. This allows discarding failed experiments without polluting history.

► Challenge 1: Branching Strategy

Collaborative Notebooks & Repro Workflows

Jupyter/Quarto notebooks are challenging for Git diffs. Use [nbdev](#)-style principles and Quarto to enforce reproducibility.

```
1 # Check for nbdev for cleaner notebook diffs
2 import subprocess
3
4 try:
5     result = subprocess.run(["nbdev", "--version"], capture_output=True, text=True)
6     print(f"nbdev version: {result.stdout.strip()} (Good for diffing notebooks)")
7 except FileNotFoundError:
8     print("nbdev not installed. Run: pip install nbdev")
9     print("This allows diffing notebook JSON vs code cells in Git.")
```

```
1 # Quarto publish automation
2 echo "render:
3   - '*.qmd'" > _quarto.yml
4 echo "Automation ready: 'quarto publish' will deploy your research docs."
```

Warning

Never commit executed notebook outputs. Use `git clean -f -X` or pre-commit hooks to strip outputs before committing. For Quarto, use `freeze: true` in the YAML to cache expensive computations.

► Challenge 2: Notebook Cleanup

Large Data & LFS Mastery

Geospatial data (NetCDF, shapefiles) is too large for standard Git. Use **Git LFS** and **DVC** (Data Version Control).

```
1 # Git LFS tracking for climate model outputs
2 git lfs track "*.nc"
3 git lfs track "*.shp"
4 git lfs track "*.tif"
5 cat .gitattributes
6
7 # DVC integration demo
8 dvc init
9 dvc add data/ethiopia_climate.nc
10 git add data/ethiopia_climate.nc.dvc .gitignore
11 git commit -m "add climate data to dvc"
```

```
1 # Geopandas workflow for Ethiopian administrative boundaries
2 import geopandas as gpd
3 from shapely.geometry import Point
4
5 # Create mock Ethiopian zones
6 zones = ['Tigray', 'Afar', 'Amhara', 'Oromia', 'Somali',
7         'Benishangul-Gumuz', 'SNNPR']
8 geometries = [Point(i, 10) for i in range(len(zones))]
9 gdf = gpd.GeoDataFrame({'zone': zones, 'geometry': geometries},
10                        crs='EPSG:4326')
11 print(gdf.head())
12 print(f"Working with {len(gdf)} zones. Using Git LFS to store shapefiles.")
```



Tip

Git LFS vs DVC: Use LFS for binary files you diff rarely (images, small models). Use **DVC** for datasets >1GB—i.e. data in cloud/S3 with only pointers in Git.

► Challenge 3: Data Versioning

GitHub Actions for DS CI/CD

Automate environment testing, linting, and deployment to **HuggingFace Spaces**.

```
1 # GitHub Actions workflow for DS
2 mkdir -p .github/workflows
3 cat << EOF > .github/workflows/ds-ci.yml
4 name: DS CI/CD
5
6 on: [push]
7
8 jobs:
9   test:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v4
13       - uses: actions/setup-python@v5
14         with:
15           python-version: '3.10'
16       - name: Install conda env
17         run: |
18           pip install mamba
19           mamba env create -f environment.yml
20       - name: Lint with black and ruff
21         run: |
22           pip install black ruff
23           black . --check --diff
```

❗ Important

For HuggingFace Spaces, use the [huggingface_hub](#) Python library or `git push` with a token stored in GitHub S (`HF_TOKEN`). This enables automatic updates of your Gradio/Streamlit app.

► Challenge 4: CI/CD Workflow

Power User Commands

Interactive rebase and stash are lifesavers for messy analysis and context switching.

```
1 # View recent commits
2 git log --oneline -3
3
4 # Interactive rebase to clean history
5 echo "To squash 3 commits into one:"
6 echo "git rebase -i HEAD~3"
7 echo "> (In editor: change 'pick' to 'squash' for last two commits)"
8
9 # Stash for interrupted analysis
10 echo "Saving current work-in-progress:"
11 echo "git stash push -m 'WIP: Climate data normalization'"
12 git stash list
13 echo "Restoring later: git stash pop"
```

```
1 # Visual representation of stash
2 git stash show -p
```



Tip

Interactive Rebase: Use *before* pushing to main. Transform "fix typo", "add data", "final fix" into a single "add Ethiopian crop model" for cleaner history.

► Challenge 5: Clean History

DS-Ready Repo Patterns

When working on High-Performance Computing clusters, use shallow clones and pre-commit hooks.

```
1 # Shallow clone (latest commit only, minimal bandwidth)
2 # git clone --depth 1 https://github.com/user/repo.git
3
4 # Pre-commit hooks configuration
5 cat << EOF > .pre-commit-config.yaml
6 repos:
7   - repo: https://github.com/psf/black
8     rev: 24.0.0
9     hooks:
10       - id: black
11   - repo: https://github.com/astral-sh/ruff-pre-commit
12     rev: v0.3.0
13     hooks:
14       - id: ruff
15   - repo: local
16     hooks:
17       - id: data-validator
18         name: Validate NetCDF headers
19         entry: python scripts/validate_netcdf.py
20         language: system
21         files: /\.nc$
22 EOF
23
24 # Mock data validator for NetCDF files
25 def validate_netcdf_mock(filepath):
26     """Check for required climate variables."""
27     print(f"Validating {filepath}")
28     required_vars = ['lat', 'lon', 'time', 'precip']
29     print(f"Mock validation passed: {required_vars} found")
30     return True
31
32 validate_netcdf_mock("data/ethiopia_era5.nc")
```

 Note

Pre-commit hooks on HPC: Run `pre-commit install --install-hooks` after cloning on a cluster. This ensure linting and prevents accidental commits of large binaries without LFS.

► **Challenge 6: Custom Validator**

GitHub Features for Research

Use **Issue Templates** for experiment tracking and **Projects** for managing paper submission timelines.

```
1 # Create Issue Template for Experiments
2 mkdir -p .github/ISSUE_TEMPLATE
3 cat << EOF > .github/ISSUE_TEMPLATE/experiment.md
4 ---
5 name: ML Experiment
6 about: Track a specific model or data experiment
7 title: '[EXP] '
8 labels: experiment
9 assignees: ''
10
11 ---
12
13 ## Hypothesis
14 *What are we testing?*
15
16 ## Metrics
17 - [ ] Baseline RMSE:
18 - [ ] New RMSE:
19
20 ## Git Branch
21 \`experiment/\`
22
23 ## Data Version
```



Tip

Projects: Link pull requests directly to project items. For a paper timeline, use a “Milestone” for the submission visualize experiments blocking publication.

► Challenge 7: Project Management

Cheat Sheet

Top 10 Git aliases for Data Scientists

Alias	Command	DS Use Case
<code>git graph</code>	<code>log --graph --oneline --all</code>	Visualize experiment branches
<code>git unstage</code>	<code>reset HEAD --</code>	Undo <code>git add</code> on a huge CSV
<code>git last</code>	<code>log -1 HEAD</code>	Check last model commit
<code>git wip</code>	<code>commit -am "WIP"</code>	Quick save before meeting
<code>git undo</code>	<code>reset --soft HEAD^</code>	Undo last commit, keep changes
<code>git lfs-ls</code>	<code>lfs ls-files</code>	List large files tracked
<code>git nb-clean</code>	<code>!jupyter nbconvert --ClearOutputPreprocessor.enabled=True %1</code>	Clean notebook before commit
<code>git sync</code>	<code>!git pull --rebase && git push</code>	Keep linear history
<code>git blame-ds</code>	<code>blame -L 10,20 notebook.ipynb</code>	Find who broke analysis
<code>git dvc-push</code>	<code>!dvc push && git push</code>	Sync data and code

`.gitconfig` snippet

Resources

Essential Tools for Data Science Git Workflows

- **DVC** - Version control for machine learning datasets and pipelines. Integrates with cloud storage (S3, GCS, Azure).
- **pre-commit** - Framework for managing multi-language pre-commit hooks. Essential for code quality and preventing data leaks.
- **nbdev** - Develop libraries in Jupyter Notebooks with automatic export, documentation, and testing.
- **Quarto** - Scientific and technical publishing system supporting Python, R, Julia, and Observable.
- **Git LFS** - Manage large files in Git repositories with pointer files and remote storage.
- **nbdime** - Smart diffing and merging for Jupyter notebooks.

Further Reading

- [GitHub Actions Documentation](#)
- [Hugging Face Spaces](#)
- [Geopandas for Ethiopian Spatial Data](#)

Summary

- Branch strategically: `main` for production, `experiment/` for ML iterations
- Use Quarto and nbdev for reproducible notebook workflows
- Manage large data with Git LFS (binaries) and DVC (datasets >1GB)
- Automate CI/CD with GitHub Actions (testing, linting, deployment)
- Master power commands: rebase for clean history, stash for context switching
- Configure pre-commit hooks for HPC cluster compatibility
- Leverage GitHub Issues and Projects for research management

Supplemental: Using Git from RStudio

Questions

- How can I use Git with RStudio?

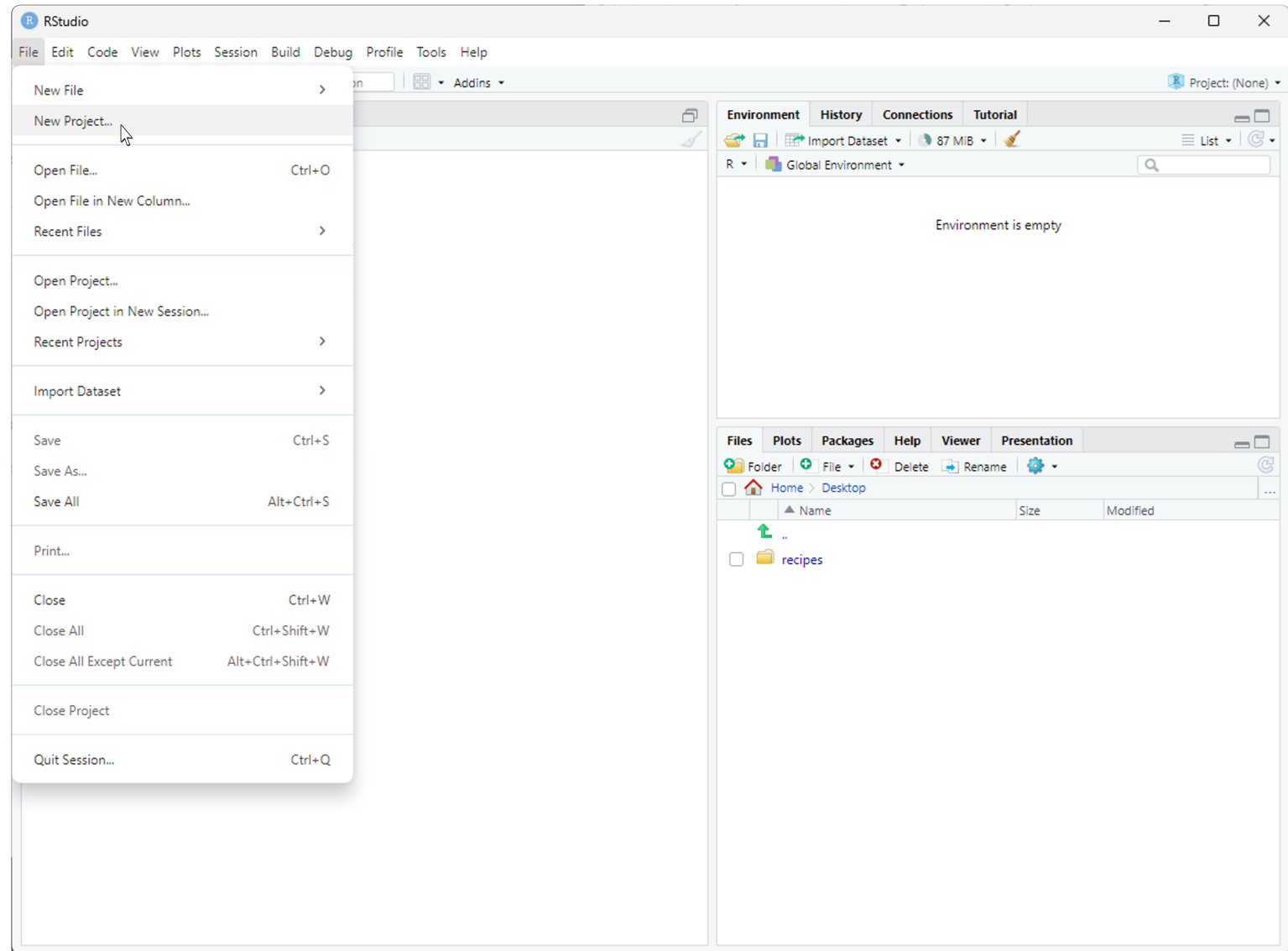
Why Use RStudio for Version Control?

Version control can be very useful when developing data analysis scripts. For that reason, the popular development environment RStudio for the R programming language has built-in integration with Git. While some advanced Git features still require the command-line, RStudio has a nice interface for many common operations.

RStudio allows us to create a project associated with a given directory to keep track of various related files. We can be able to track the development of the project over time, to be able to revert to previous versions, and to collaborate with others, we version control the RStudio project with Git.

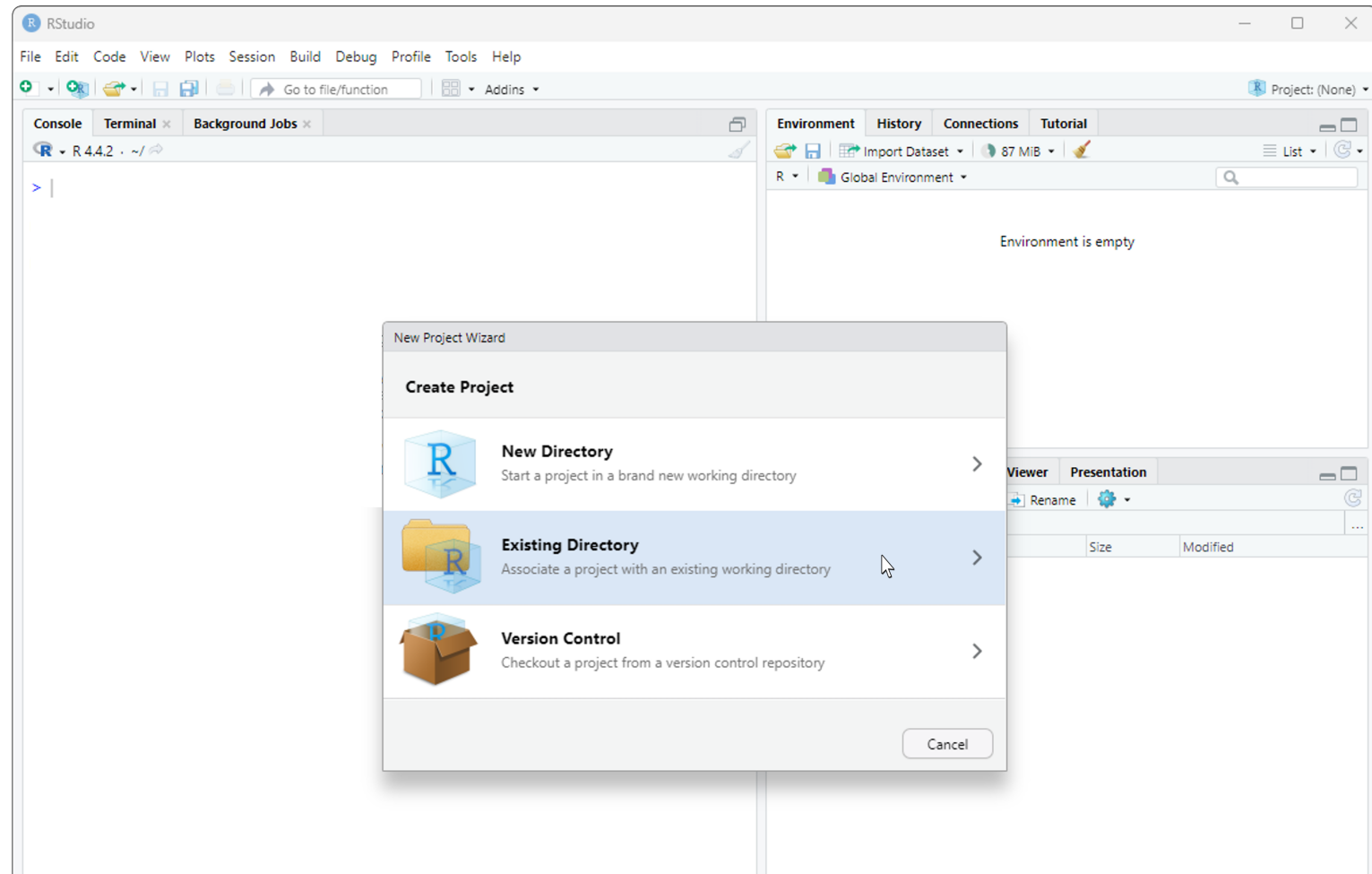
Creating a New Project

To get started using Git in RStudio, we create a new project:



RStudio New Project

This opens a dialog asking us how we want to create the project. We have some options here. Let's say tl want to use RStudio with the **Malaria** repository that we already made. Since that repository lives in a directory on our computer, we choose the option "Existing Directory":



Setting Up Git in RStudio

Do You See a “Version Control” Option?

Although we’re not going to use it here, there should be a “version control” option on this menu. That is what you would click on if you wanted to create a project on your computer by cloning a repository from GitHub. If that option is not present, it probably means that RStudio doesn’t know where your Git executable is.

Find your Git Executable

First let’s make sure that Git is installed on your computer. Open your shell or command prompt and then type:

- `which git` (macOS, Linux)
- `where git` (Windows)

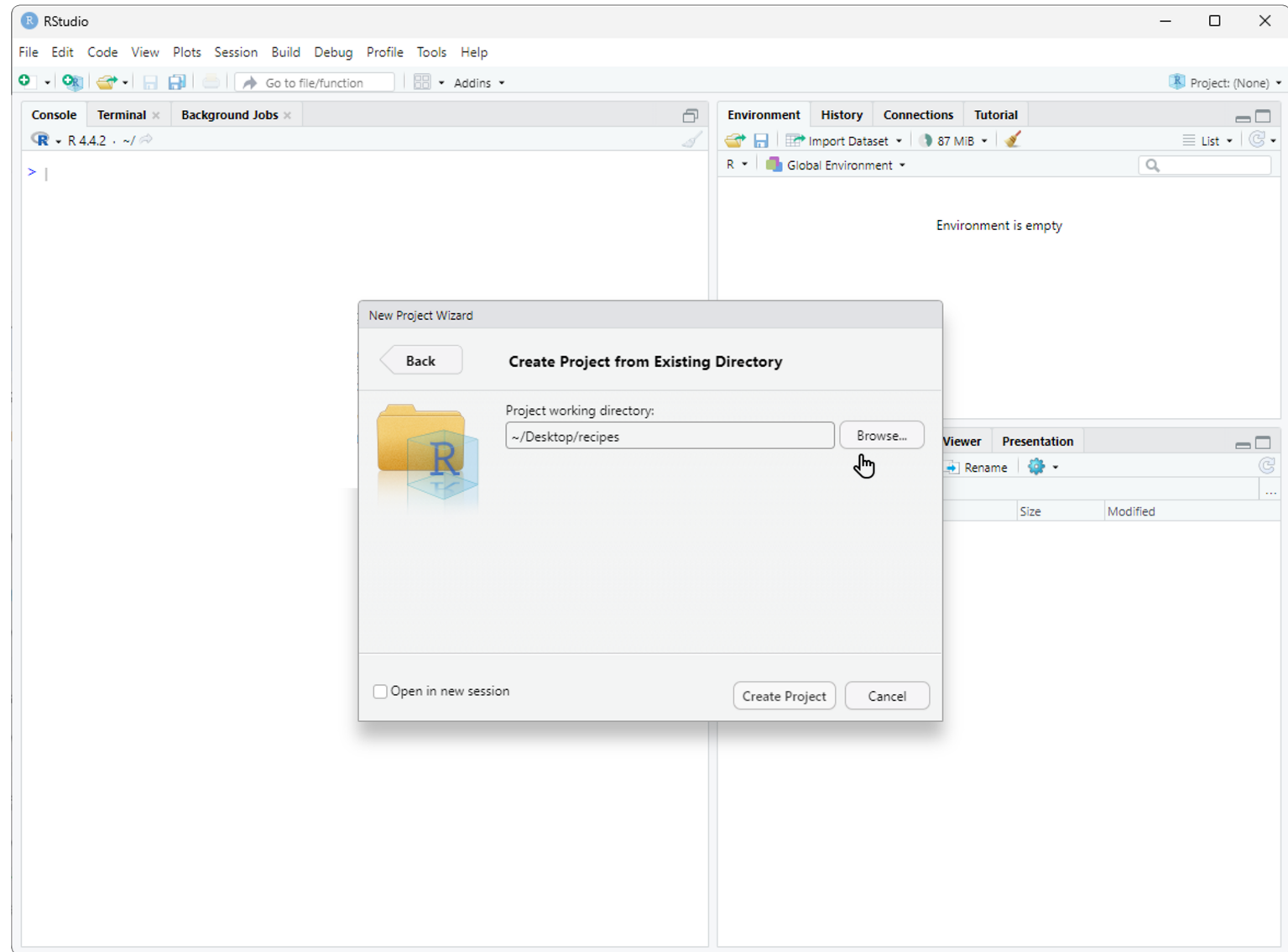
If there is no version of Git on your computer, please follow the Git installation instructions to install Git now.

Tell RStudio where to find Git

In RStudio, go to the **Tools** menu > **Global Options** > **Git/SVN** and then browse to the Git executable you found. Then restart RStudio.

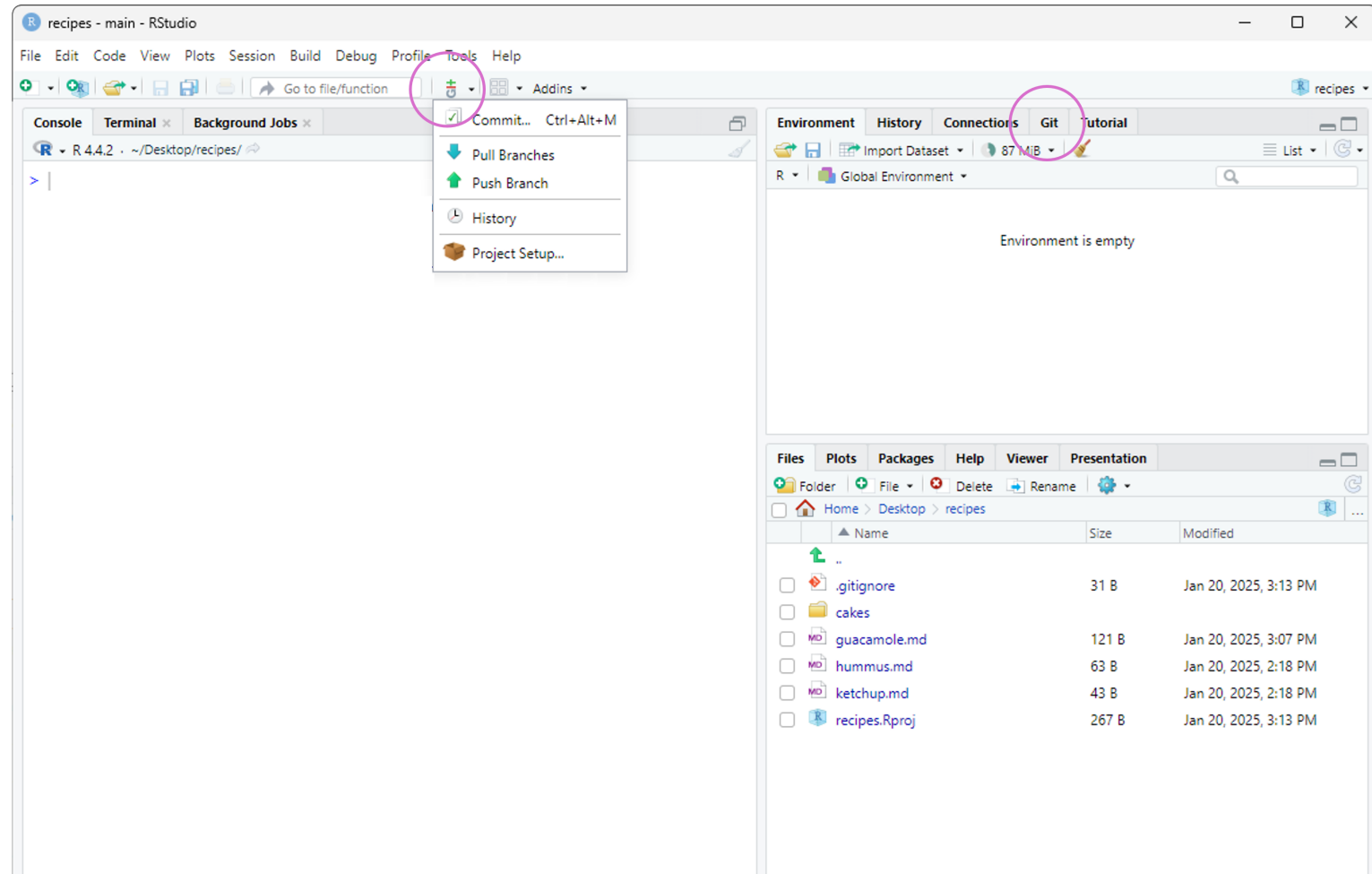
Opening Existing Repository

Next, RStudio will ask which existing directory we want to use. Click “Browse...” and navigate to the correct directory, then click “Create Project”:



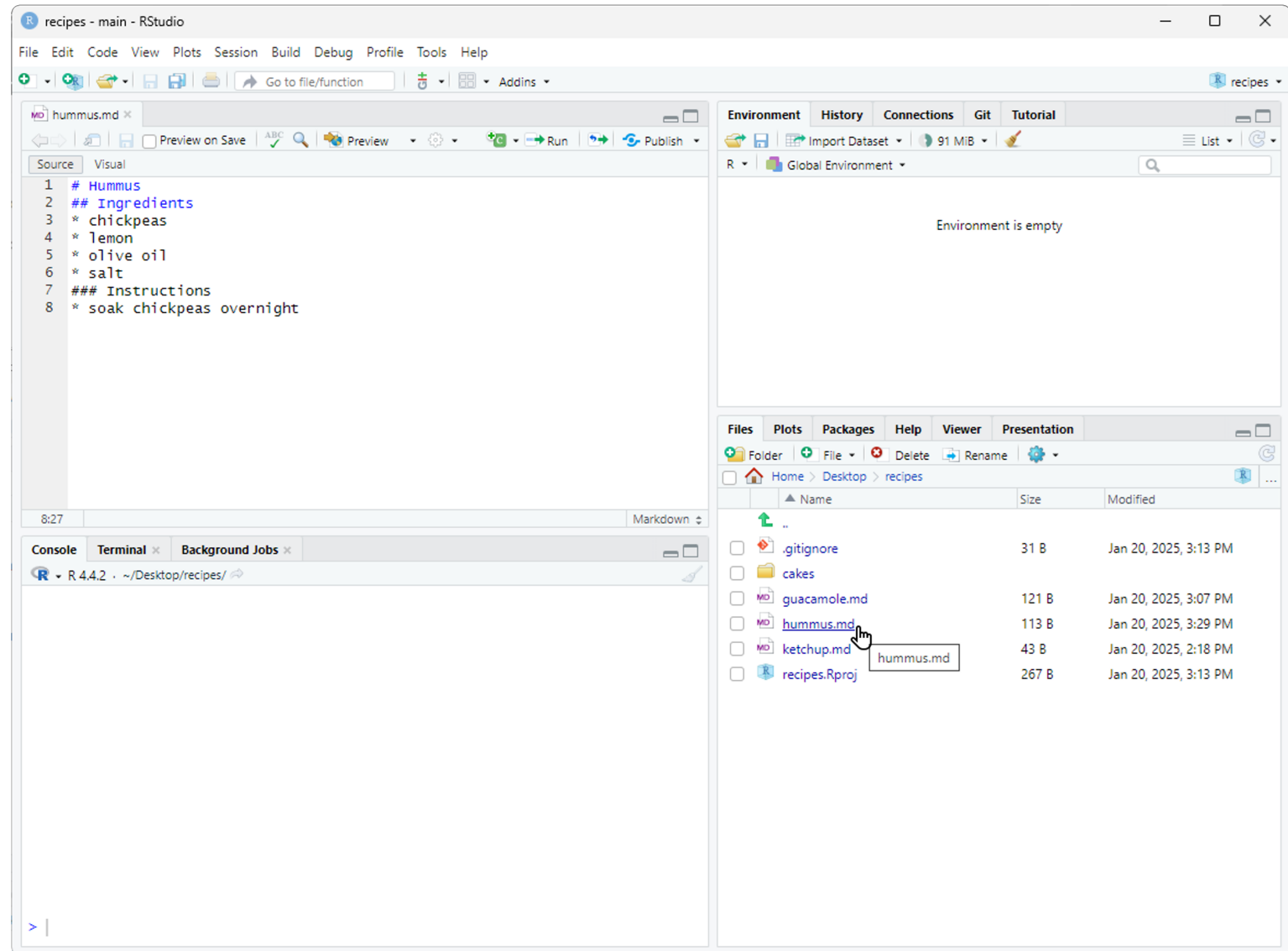
Navigate to Existing Directory

Ta-da! We have created a new project in RStudio within the existing **Malaria** repository. Notice the vertical “Git” menu in the menu bar. RStudio has recognized that the current directory is a Git repository, and gives a number of tools to use Git:



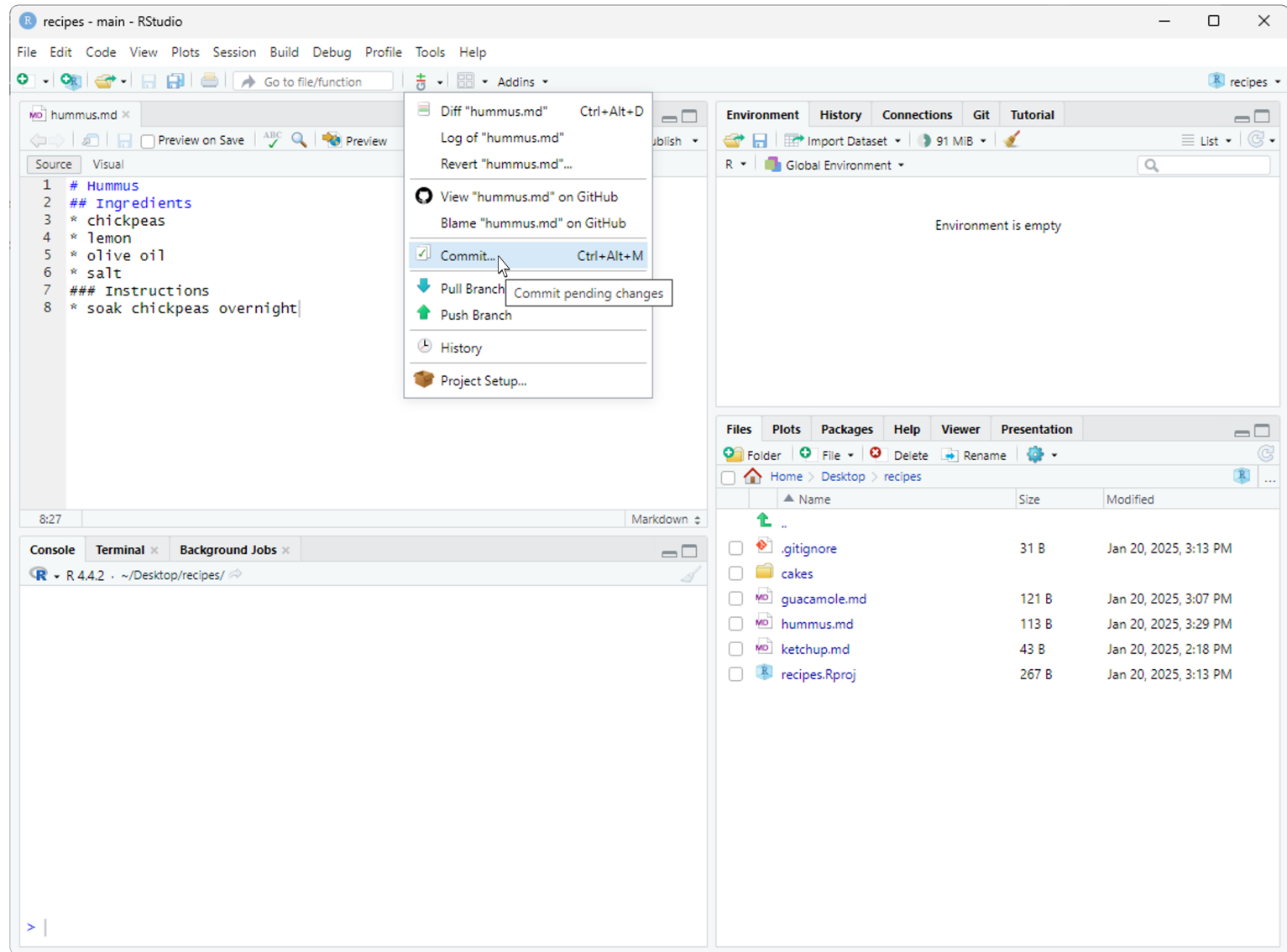
Editing Files

To edit the existing files in the repository, we can click on them in the “Files” panel on the lower right. Now add some additional information about Malaria data analysis:



Editing Files

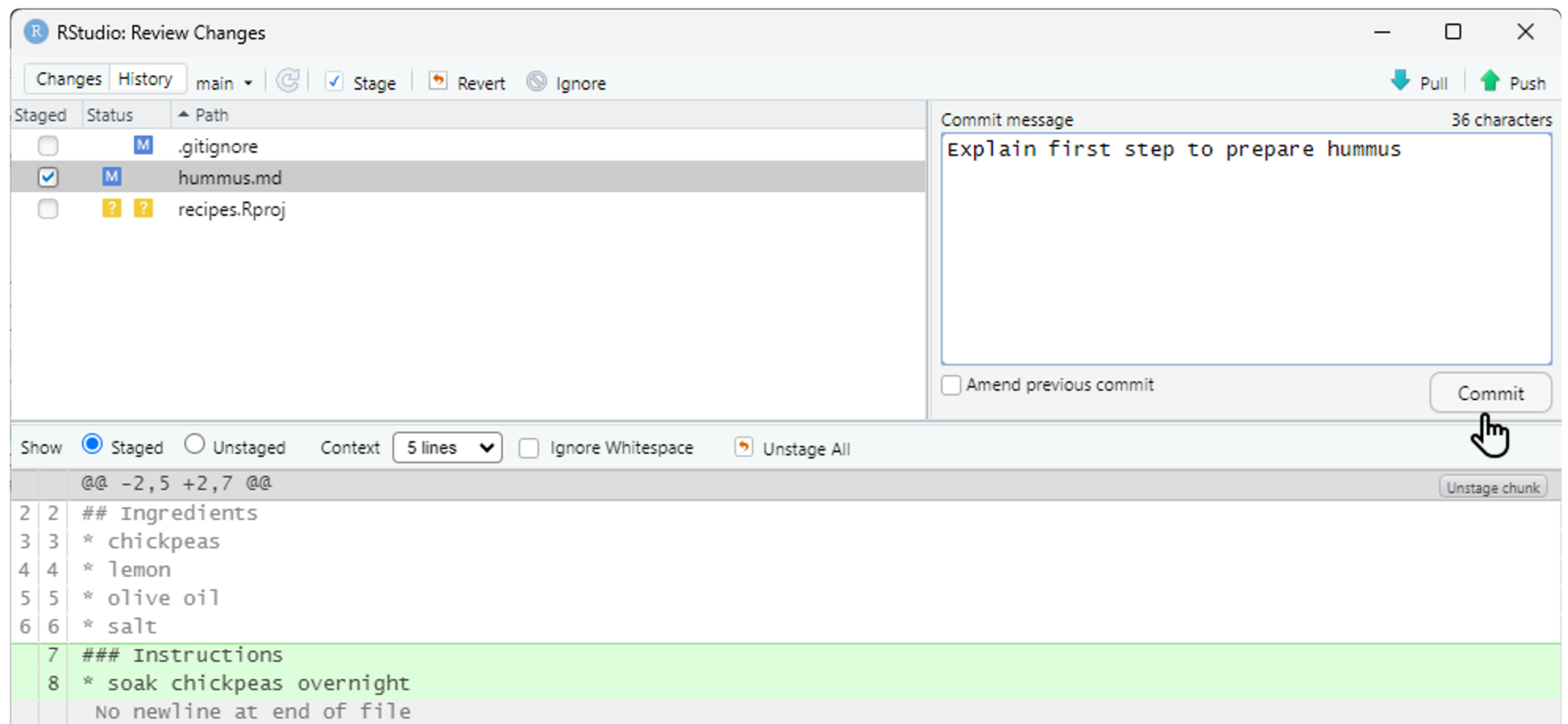
Once we have saved our edited files, we can use RStudio to commit the changes by clicking on “Commit to the Git menu:



Commit Menu

Reviewing Changes

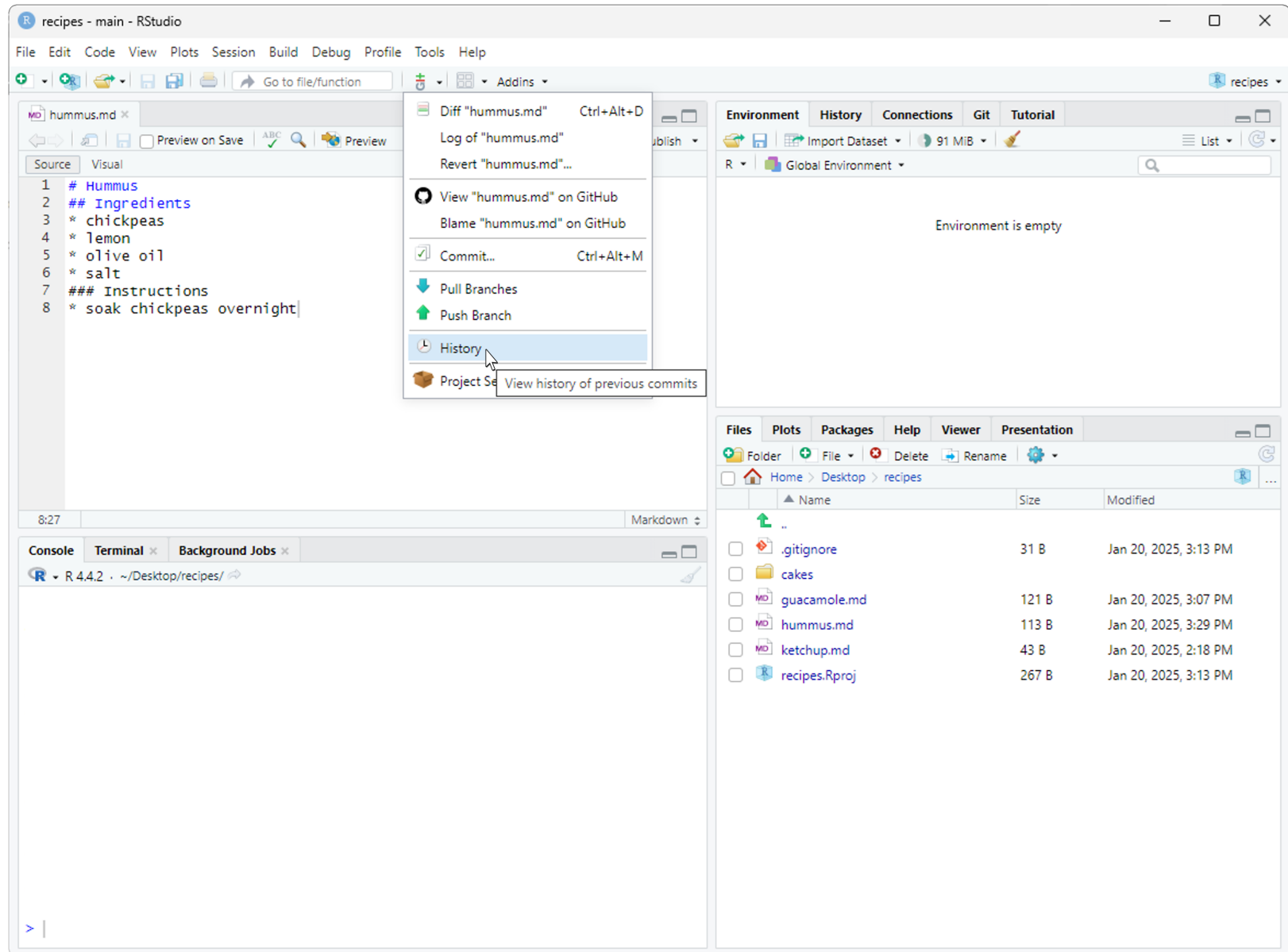
This will open a dialogue where we can select which files to commit (by checking the appropriate boxes in the “Staged” column), and enter a commit message (in the upper right panel). The icons in the “Status” column indicate the current status of each file. Clicking on a file shows information about changes in the lower panel (using output of `git diff`). Once everything is the way we want it, we click “Commit”:



Review Changes

Pushing and Pulling

The changes can be pushed by selecting “Push Branch” from the Git menu. There are also options to pull from the remote repository, and to view the commit history:



Git Menu Options

Viewing History

If we click on “History”, we can see a graphical version of what `git log` would tell us:

RStudio: Review Changes

Changes History main (all commits) Search Pull

Subject	Author	Date (UTC)	SHA
HEAD -> refs/heads/main origin/main origin/HEAD Explain first step to prepare hummus	Alfredo Linguini <a.linguini@ratatouille.fr>	2025-01-20	ce5431c6
Merge changes from GitHub	Alfredo Linguini <a.linguini@ratatouille.fr>	2025-01-20	228dea3f
Add first step	Alfredo Linguini <a.linguini@ratatouille.fr>	2025-01-20	ee647906
First step on the instructions	Jimmy <a.collaborator@gmail.com>	2025-01-20	5986ac9d
Add ingredients for hummus	Jimmy <a.collaborator@gmail.com>	2025-01-20	ca97aa3a
Ignore png file and the receipts folder	Alfredo Linguini <a.linguini@ratatouille.fr>	2024-11-29	0fc109b5
My opinions about the red sauce	Alfredo Linguini <a.linguini@ratatouille.fr>	2024-11-29	d74eafe1
Add some initial cakes	Alfredo Linguini <a.linguini@ratatouille.fr>	2024-11-28	10db591d
Modify guacamole to the traditional recipe	Alfredo Linguini <a.linguini@ratatouille.fr>	2024-11-28	c63abbbe
Add basic guacamole's ingredients	Alfredo Linguini <a.linguini@ratatouille.fr>	2024-11-28	4e44c9ad
Create a template for recipe	Alfredo Linguini <a.linguini@ratatouille.fr>	2024-11-28	d8b82a1e

Commits 1-11 of 11

SHA ce5431c61117ceb34ae34b7503b3d013bf0839ca
Author Alfredo Linguini <a.linguini@ratatouille.fr>
Date (UTC) 2025-01-20 05:41
Subject Explain first step to prepare hummus
Parent 228dea3f0254cfd115ba4f8474a9a31b5dba46a2

hummus.md

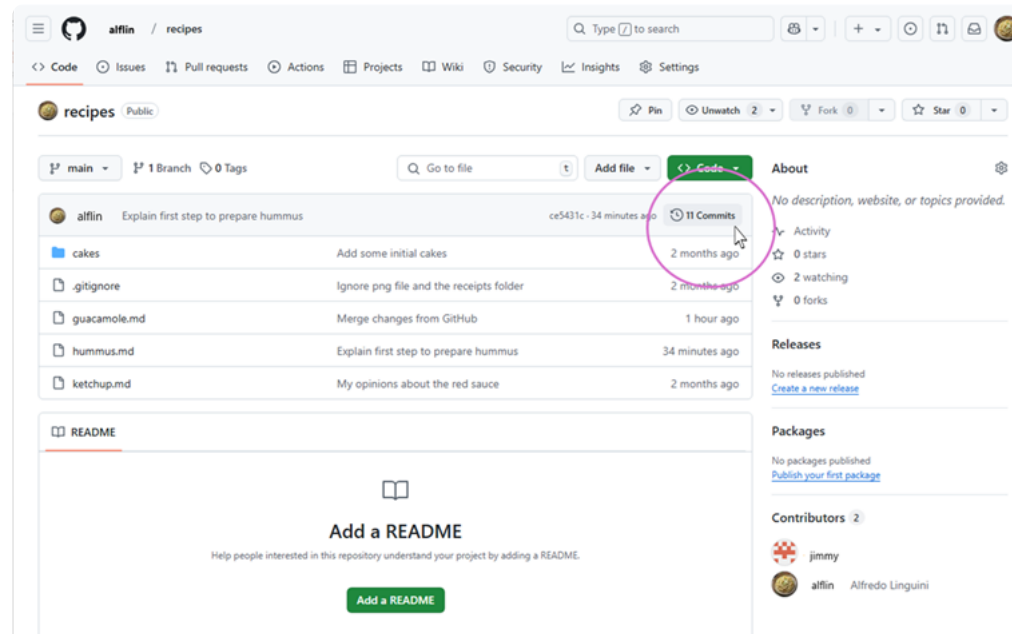
hummus.md View file @ ce5431c6

```
@@ -4,3 +4,5 @@
4 4 * lemon
5 5 * olive oil
6 6 * salt
7 ### Instructions
8 * soak chickpeas overnight
No newline at end of file
```

View History

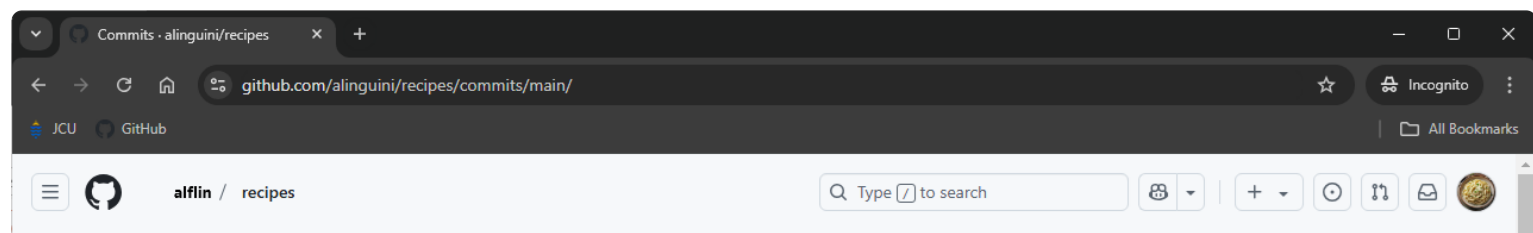
GitHub Commit History

To view it, go to your repository page and click on the “Commits” link near the top:



GitHub Commit History Button

This will open a list of all commits, showing who made each change and when:



[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

Commits

main

All usersAll time

Commits on Jan 20, 2025

Explain first step to prepare hummus

alflin committed 32 minutes ago

ce5431c

Merge changes from GitHub

alflin committed 1 hour ago

228dea3

Add first step

alflin committed 1 hour ago

ee64790

First step on the instructions

jimmy committed 1 hour ago

5986ac9

Add ingredients for hummus

jimmy committed 1 hour ago

ca97aa3

Commits on Nov 29, 2024

Ignore png file and the receipts folder

alflin committed on Nov 29, 2024

0fc109b

My opinions about the red sauce

alflin committed on Nov 29, 2024

d74eafe

Add some initial cakes

alflin committed on Nov 29, 2024

10db591

Modify guacamole to the traditional recipe

alflin committed on Nov 29, 2024

c63abbb

Add basic guacamole's ingredients

alflin committed on Nov 29, 2024

4e44c9a

Create a template for recipe

alflin committed on Nov 29, 2024

d8b82a1

GitHub Commit History

After seeing how your commit history appears on GitHub, you can be confident that your changes have successfully pushed and recorded.

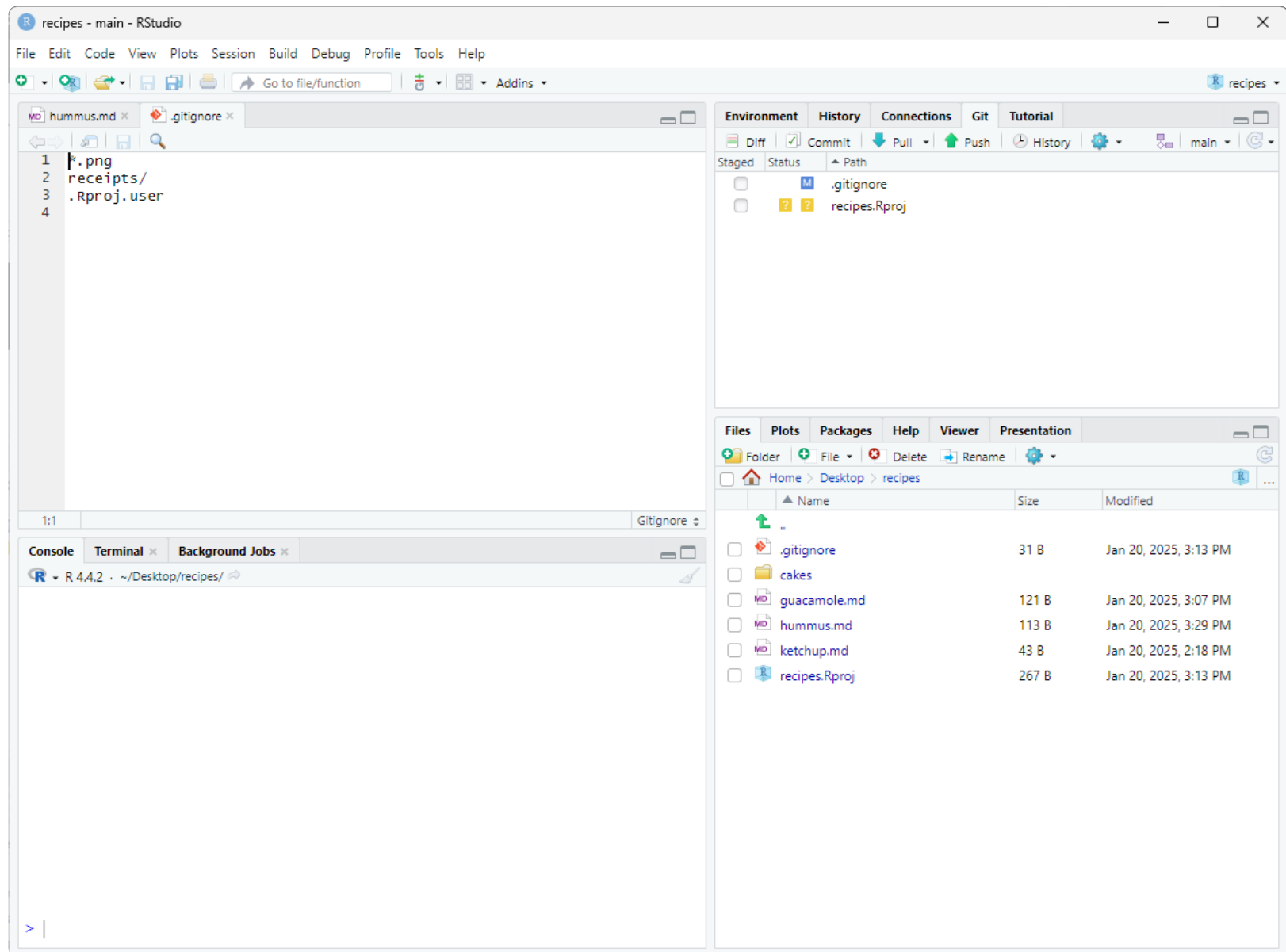
RStudio and .gitignore

Next, let's take a look at how RStudio quietly helps manage your repository by automatically updating your `.gitignore` file.

When you create a *New Project* in RStudio, it generates an `.Rproj` file and a hidden folder called `.Rproj.user`. These are used to store project-specific settings and user preferences.

RStudio recognizes that these files typically shouldn't be tracked in version control, so it automatically adds them to your existing `.gitignore` file.

Notice that the `.gitignore` file was modified and appears as a changed file in the Git tab on the right-hand side panel:



Gitignore Update

Challenge: Push .gitignore Changes

Consider adding other files to be ignored, such as `Malaria.Rproj`, `.Rhistory`, and `.RData`, and complete the process for saving RStudio's `.gitignore` file changes to your remote repository.

► [View Solution](#)

RStudio Git Integration Summary

There are many more features in the RStudio Git menu, including:

- **Diff viewer** - See changes between versions
- **Blame** - See who last modified each line of code
- **Branch management** - Create and switch between branches
- **Merge tools** - Resolve conflicts graphically

These tools make it easier for public health researchers and data scientists to incorporate version control their daily workflow without needing to memorize complex command-line operations.

Resources

Resource	URL	Description
Pro Git Book	https://git-scm.com/book	Complete Git reference
Git Flight Rules	https://github.com/k88hudson/git-flight-rules	Troubleshooting guide
Atlassian Git Tutorials	https://www.atlassian.com/git/tutorials	Visual tutorials
DVC Documentation	https://dvc.org/doc	Data version control
GitHub Skills	https://skills.github.com	Interactive tutorials

Thank you!



References

- Pro Git Book: <https://git-scm.com/book>
- Git Flight Rules: <https://github.com/k88hudson/git-flight-rules>
- Atlassian Git Tutorials: <https://www.atlassian.com/git/tutorials>
- DVC Documentation: <https://dvc.org/doc>
- GitHub Skills: <https://skills.github.com>